**RÉPUBLIQUE DE MADAGASCAR**
**UNIVERSITÉ DE FIANARANTSOA**
**ÉCOLE NATIONALE D'INFORMATIQUE**

Centre d'étude de Toliara

RAPPORT DE STAGE EN DEUXIÈME ANNÉE DE LICENCE PROFESSIONNELLE
Mention : Informatique

Parcours : Informatique Générale

Intitulé :

# CONCEPTION ET DÉVELOPPEMENT D'UNE APPLICATION DE GESTION DU PARC INFORMATIQUE (GPI)

Cas de la Direction Générale de la Sécurité Routière (DGSR)

---

Présenté le : .....

**Par : Monsieur RAKOTOMAMINIRIANA Fanomezaniavo Fenohery Sarobidy**

Membres du jury :
Rapporteurs :
- ..... (Encadreur pédagogique)
- ..... (Encadreur professionnel)

**Année universitaire 2025 - 2026**

Tampon et signature du responsable

# CURRICULUM VITAE

## RAKOTOMAMINIRIANA FANOMEZANIAVO FENOHERY SAROBIDY


- **Âge** : 23 ans
- **Situation matrimoniale** : Célibataire
- **Téléphone** : +261 34 28 752 34
- **E-mail** : rfanomezaniavo@gmail.com
- **Portfolio** : https://fenohery-pf-00.vercel.app
- **Adresse** : Maninday


### *PERSONNALITÉS*

Je suis passionné par le monde du numérique et par tout ce qui s'y rapporte. J'aime apprendre, je suis dynamique et sérieux, et je m'attache à mener à terme les missions qui me sont confiées.

### *DIPLÔMES ET FORMATIONS*

- **Depuis 2023** : deuxième année de Licence à l'École Nationale d'Informatique de Fianarantsoa, centre d'étude de Toliara.
- **2020 – 2023** : Mathématiques et Informatique à la Faculté des Sciences d'Ankatso.
- **2021 – 2022** : Anglais à l'English Teaching Program (ETP) d'Antsahavola.
- **2019 – 2020** : Baccalauréat série C à l'École Sacré-Cœur d'Anosisoa.

### *EXPÉRIENCES PROFESSIONNELLES*

- **Décembre 2024 – Février 2025** : développeur web front-end chez SAHA Technology.
- **Septembre 2025 – Novembre 2025** : développeur d'applications avec Odoo chez ENEAM à Ivato.
- **2025 – 2026** : stage de fin de deuxième année de Licence au sein de la Direction Générale de la Sécurité Routière (DGSR), service informatique.

### *LANGUES*

- **Malgache** : langue maternelle.
- **Français** : parler, lire, écrire.
- **Anglais** : parler, lire, écrire.

## *COMPÉTENCES*

- **Technologies web et design** : React, Tailwind CSS, ExpressJS, NodeJS, Laravel, Figma.
- **Langages de programmation** : C/C++, JavaScript, PHP, Python.
- **Systèmes de gestion de bases de données** : MySQL, SQLite et PostgreSQL.
- **Environnements de développement et collaboratifs** : Git/GitHub, Docker et Vagrant.
- **Réseaux informatiques** : TCP/IP, modèle ISO et routage IP (RIP et OSPF).
- **Informatique bureautique** : Word, Excel et PowerPoint.
- **Systèmes** : Windows et Unix/Linux.

## *CENTRES D'INTÉRÊT*

- Technologie
- Jeux vidéo
- Films
- Musique
- Jeux d'échecs

# SOMMAIRE

Le sommaire présente les parties, les chapitres et les titres de premier niveau du rapport. La table des matières, placée en fin de document, en donne le détail complet.


CURRICULUM VITAE ..... I
REMERCIEMENTS ..... V
LISTE DES FIGURES ..... VII
LISTE DES TABLEAUX ..... VIII
LISTE DES ABRÉVIATIONS ..... X
INTRODUCTION GÉNÉRALE ..... 1

**PARTIE I. PRÉSENTATIONS ..... 3**

Chapitre 1. Présentation de l'ENI ..... 4
1.1. Informations d'ordre général ..... 4
1.2. Missions et historique ..... 4
1.3. Organigramme institutionnel ..... 6
1.4. Domaines de spécialisation ..... 6
1.5. Architecture des formations pédagogiques ..... 7
1.6. Relations de l'ENI avec les organismes externes ..... 8
1.7. Débouchés professionnels ..... 8
1.8. Ressources humaines ..... 9
Chapitre 2. Présentation de la DGSR ..... 10
2.1. Statut et missions ..... 10
2.2. Organisation administrative ..... 10
2.3. Organigramme fonctionnel ..... 11
2.4. Valeurs et engagements ..... 11
2.5. Le Service Informatique ..... 12
Chapitre 3. Description du projet ..... 13
3.1. Formulation ..... 13
3.2. Objectif et besoins des utilisateurs ..... 13
3.3. Moyens nécessaires à la réalisation du projet ..... 14
3.4. Résultats attendus ..... 15
3.5. Chronogramme de travail ..... 15

**PARTIE II. ANALYSE ET CONCEPTION ..... 17**

Chapitre 4. Analyse préalable ..... 18
4.1. Analyse de l'existant ..... 18
4.2. Critique de l'existant ..... 19
4.3. Conception avant-projet ..... 19
Chapitre 5. Analyse conceptuelle ..... 24
5.1. Présentation de la méthode MERISE ..... 24
5.2. Dictionnaire des données ..... 25
5.3. Règles de gestion ..... 28
5.4. Diagramme de flux ..... 28
5.5. Modélisation des données ..... 30

**PARTIE III. RÉALISATION ..... 39**

Chapitre 6. Mise en place de l'environnement de développement ..... 40
6.1. Installation et configuration des outils ..... 40
6.2. Architecture de l'application ..... 43
6.3. Structure du projet ..... 44
6.4. Déploiement en mode intranet ..... 45
Chapitre 7. Développement de l'application ..... 47
7.1. Création de la base de données ..... 47

7.2. Codage de l'application ..... 48
7.3. Présentation de l'application ..... 53
7.4. Perspectives d'évolution ..... 64
CONCLUSION GÉNÉRALE ..... 66
BIBLIOGRAPHIE ..... 68
WEBOGRAPHIE ..... 69
GLOSSAIRE ..... 70
ANNEXES ..... 71
Annexe A. Extrait du fichier de routes de l'API ..... 71
Annexe B. Extrait du script de migration de la table equipements ..... 72
RÉSUMÉ ..... 76
ABSTRACT ..... 77

# REMERCIEMENTS

Avant de commencer, je remercie Dieu tout-puissant, sans qui rien de tout cela n'aurait été possible. Mes remerciements s'adressent également à celles et ceux qui ont rendu ce stage possible.

– À **Monsieur HAJALALAINA Aimé Richard**, Professeur, Président de l'Université de Fianarantsoa, pour son engagement en faveur de l'excellence académique et son soutien aux étudiants.

– À **Monsieur ANDRIAMANANTENA Razafiharison**, Président de l'Université de Toliara, pour avoir accordé l'intégration du centre d'étude de l'ENI au sein de l'Université de Toliara.

– Au **Docteur MAHATODY Thomas**, Directeur de l'École Nationale d'Informatique de Fianarantsoa, pour la qualité de la formation dispensée et son attention aux conditions de stage des étudiants.

– À **Monsieur le Directeur Général de la Direction Générale de la Sécurité Routière**, pour avoir accepté ma candidature et m'avoir accueilli au sein de son établissement durant toute la période du stage.

– À **Monsieur RABETAFIKA Louis Haja**, Maître de Conférences, responsable de la mention Informatique, pour ses conseils méthodologiques et son suivi de la formation.

– À **Monsieur GILANTE Gesazafy**, responsable du parcours Informatique Générale, pour son accompagnement tout au long de l'année universitaire.

– À **Monsieur RAZAFINDRAMONJA Herinantenaina Clément Aubert**, Assistant d'Enseignement Supérieur et de Recherche à l'ENI, mon encadreur pédagogique et rapporteur, pour ses orientations, sa rigueur méthodologique et ses relectures attentives de ce rapport.

– À **Monsieur RABENJARISOA Tantelinirina**, responsable du Service Informatique de la DGSR et mon encadreur professionnel, pour m'avoir guidé dans la réalisation du projet, pour sa disponibilité et pour la confiance qu'il m'a accordée.

– À l'ensemble du **Service Informatique de la DGSR**, pour son accueil, ses explications sur l'organisation du parc et sa participation aux tests de l'application.

– À tous les **corps enseignants** de l'ENI, pour leur patience et leur engagement malgré leurs charges académiques.

– À mes **parents** et à tous mes **proches**, pour leur soutien, financier comme moral, et pour leurs prières.

# LISTE DES FIGURES

Figure 1. Modèle Conceptuel des Données (MCD) du système GPI ..... 31
Figure 2. Modèle Logique des Données (MLD) du système GPI ..... 35
Figure 3. Modèle Conceptuel des Traitements (MCT) — tickets, affectations et inventaire ..... 36
Figure 4. Modèle Conceptuel des Traitements (MCT) — restitution et réforme ..... 37
Figure 5. Modèle Organisationnel des Traitements (MOT) du système GPI ..... 38
Figure 6. Architecture trois tiers de l'application GPI ..... 43
Figure 7. Page d'authentification de l'application GPI ..... 54
Figure 8. Tableau de bord : indicateurs et répartition du parc ..... 55
Figure 9. Liste des équipements avec recherche et filtres ..... 56
Figure 10. Fiche détaillée d'un équipement et historique ..... 57
Figure 11. Gestion des affectations en cours ..... 58
Figure 12. Suivi des interventions de maintenance ..... 59
Figure 13. Campagnes d'inventaire du parc ..... 60
Figure 14. Gestion des achats et des fournisseurs ..... 61
Figure 15. Catalogue logiciel et suivi des licences ..... 62
Figure 16. Répertoire des agents de l'établissement ..... 63
Figure 17. Rapports et historiques du parc ..... 64

# LISTE DES TABLEAUX

Tableau 1. Organes de gouvernance de l'ENI ..... 6
Tableau 2. Organisation du système de formation ..... 6
Tableau 3. Mentions et parcours au sein de l'ENI ..... 7
Tableau 4. Liste des formations existantes à l'ENI ..... 7
Tableau 5. Débouchés éventuels des jeunes diplômés ..... 8
Tableau 6. Directions et services de la DGSR ..... 11
Tableau 7. Organigramme fonctionnel de la DGSR ..... 11
Tableau 8. Caractéristiques du poste de développement ..... 14
Tableau 9. Logiciels utilisés et versions ..... 14
Tableau 10. Résultats attendus par profil d'utilisateur ..... 15
Tableau 11. Diagramme de Gantt du travail pendant treize semaines ..... 16
Tableau 12. Avantages et inconvénients du système actuel ..... 19
Tableau 13. Comparaison des solutions envisagées ..... 20
Tableau 14. Comparaison méthodologique entre MERISE et Agile ..... 20
Tableau 15. Tableau comparatif entre SGBDR ..... 21
Tableau 16. Comparaison des frameworks back-end ..... 21
Tableau 17. Comparaison des technologies front-end ..... 22
Tableau 18. Cycle d'abstraction MERISE ..... 24
Tableau 19. Dictionnaire des données — entité AGENT ..... 25
Tableau 20. Dictionnaire des données — entité ÉQUIPEMENT ..... 25
Tableau 21. Dictionnaire des données — entité AFFECTATION ..... 26
Tableau 22. Dictionnaire des données — entité TICKET ..... 26
Tableau 23. Dictionnaire des données — entité MAINTENANCE ..... 27
Tableau 24. Dictionnaire des données — entité ACHAT ..... 27
Tableau 25. Dictionnaire des données — entité LOGICIEL ..... 27
Tableau 26. Règles de gestion du système ..... 28
Tableau 27. Acteurs du système ..... 29
Tableau 28. Traitements, acteurs et ressources du système ..... 38
Tableau 29. Principales tables de la base GPI ..... 47
Tableau 30. Principaux points d'accès de l'API ..... 49
Tableau 31. Rôles et permissions de l'application ..... 50
Tableau 32. Cas de tests et résultats ..... 52

Tableau 33. Difficultés rencontrées et solutions ..... 53

# LISTE DES ABRÉVIATIONS

| Sigle | Signification |
|---|---|
| API | Application Programming Interface (interface de programmation applicative) |
| CENSERO | Centre National de Sécurité Routière |
| CLI | Command Line Interface (interface en ligne de commande) |
| CRT | Centre de Réception Technique |
| CRUD | Create, Read, Update, Delete |
| DAF | Direction Administrative et Financière |
| DG | Direction Générale |
| DGSR | Direction Générale de la Sécurité Routière |
| DOR | Direction des Opérations Routières |
| DT | Direction Technique |
| ENI | École Nationale d'Informatique |
| GPI | Gestion du Parc Informatique |
| HTTP(S) | HyperText Transfer Protocol (Secure) |
| IDE | Integrated Development Environment |
| JSON | JavaScript Object Notation |
| LMD | Licence – Master – Doctorat |
| MCD | Modèle Conceptuel des Données |
| MCF | Modèle Conceptuel des Flux |
| MCT | Modèle Conceptuel des Traitements |
| MERISE | Méthode d'Étude et de Réalisation Informatique pour les Systèmes d'Entreprise |
| MLD | Modèle Logique des Données |
| MLT | Modèle Logique des Traitements |
| MOD | Modèle Organisationnel des Données |
| MOT | Modèle Organisationnel des Traitements |
| MPD | Modèle Physique des Données |
| MVC | Modèle – Vue – Contrôleur |
| ORM | Object-Relational Mapping (mapping objet-relationnel) |
| PDF | Portable Document Format |
| PHP | PHP: Hypertext Preprocessor |
| QR | Quick Response (code-barres bidimensionnel) |
| RG | Règle de gestion |
| SGBD | Système de Gestion de Base de Données |
| SGBDR | Système de Gestion de Base de Données Relationnelle |
| SPA | Single Page Application (application web mono-page) |
| SQL | Structured Query Language |
| TIC | Technologies de l'Information et de la Communication |
| UI | User Interface (interface utilisateur) |
| UX | User Experience (expérience utilisateur) |

# INTRODUCTION GÉNÉRALE

Dans toute organisation, le parc informatique représente à la fois un investissement financier important et un outil de travail quotidien. Ordinateurs, serveurs, imprimantes, équipements réseau et logiciels conditionnent le fonctionnement des services : dès qu'un matériel est mal inventorié, mal affecté ou mal suivi, ce sont les activités de l'établissement tout entier qui ralentissent. La gestion du parc informatique est donc devenue un enjeu de gouvernance, de sécurité et de maîtrise des coûts.

La Direction Générale de la Sécurité Routière (DGSR) n'échappe pas à cette réalité. Son parc informatique est réparti entre le siège et les structures déconcentrées (centres de réception technique, CENSERO), entre plusieurs directions et services, et entre des agents qui changent de poste, de bureau ou d'affectation. Le suivi de ce parc repose encore largement sur des fichiers tableurs, des registres papier et des échanges de courriels : les informations sont dispersées, parfois contradictoires, et il est difficile de savoir avec certitude quel matériel est en service, en stock, en maintenance ou réformé, ni à quel agent il est affecté.

C'est dans ce contexte que s'inscrit mon stage de deuxième année de Licence Professionnelle, effectué au sein du Service Informatique de la DGSR. L'objectif principal est la conception et le développement d'une application web intranet de **gestion du parc informatique (GPI)**. Cette plateforme doit centraliser l'inventaire des équipements, suivre leurs affectations et leurs mouvements, organiser les maintenances et les tickets d'assistance, gérer les achats et les fournisseurs, recenser les logiciels et leurs licences, et produire des rapports et des exports PDF destinés aux responsables.

Pour répondre aux exigences fonctionnelles et techniques du projet, j'ai mis en œuvre une architecture web moderne et complémentaire :

- **Laravel**, framework PHP, pour la partie serveur : il expose une API REST, gère l'authentification, les rôles et les règles de gestion, et s'appuie sur l'ORM Eloquent pour dialoguer avec la base de données.
- **React**, bibliothèque JavaScript, pour la partie cliente : elle fournit une interface réactive et ergonomique, organisée en pages et en composants réutilisables.
- **MySQL**, système de gestion de base de données relationnelle, choisi pour sa fiabilité, sa simplicité d'administration et son intégration naturelle avec l'écosystème PHP.

- **Vite** et **Tailwind CSS** pour la compilation des assets et la mise en forme de l'interface, **Git** pour le suivi des versions et **Visual Studio Code** comme environnement de développement.

Ce rapport retrace les différentes étapes de la conception de cette plateforme, depuis l'analyse des besoins jusqu'à la mise en œuvre technique, en passant par les choix technologiques, les contraintes rencontrées et les solutions apportées.

Il est organisé en trois parties. La **première partie** présente le cadre du stage : l'ENI, la DGSR et la description du projet. La **deuxième partie** expose l'analyse de l'existant et la conception du système selon la méthode MERISE (dictionnaire des données, règles de gestion, diagrammes de flux, MCD, MLD, MCT et MOT). La **troisième partie** détaille la réalisation : mise en place de l'environnement, architecture de l'application, développement, présentation des interfaces, tests et perspectives.

# PARTIE I. PRÉSENTATIONS

Cette première partie situe le cadre du stage. Elle présente successivement l'établissement de formation, l'École Nationale d'Informatique, l'organisme d'accueil, la Direction Générale de la Sécurité Routière, puis le projet lui-même : son origine, ses objectifs, ses moyens et ses résultats attendus.

# Chapitre 1. Présentation de l'ENI

L'ENI, connue sous le nom d'École Nationale d'Informatique, est située au cœur de Fianarantsoa. C'est un établissement public où l'on forme des techniciens supérieurs, des licenciés et des ingénieurs dans le domaine de l'informatique.

## 1.1. Informations d'ordre général

L'École Nationale d'Informatique est un établissement d'enseignement supérieur rattaché académiquement et administrativement à l'Université de Fianarantsoa. Elle siège à Tanambao – Antaninarenina, à Fianarantsoa. Les coordonnées pour une prise de contact avec l'école sont :

- Antaninarenina Tanambao, BP 1487 Fianarantsoa (301) ;
- Téléphone : +261 34 05 733 36 ou +261 33 42 302 02 ;
- Messagerie : eni@eni-univ-fianar.mg ;
- Site web : www.eni-univ-fianar.mg.

## 1.2. Missions et historique

L'ENI se positionne sur l'échiquier socio-éducatif malgache comme le principal secteur de diffusion et de vulgarisation des connaissances et des technologies informatiques. Cette école supérieure peut être considérée aujourd'hui comme la vitrine et la pépinière des élites informaticiennes du pays.

De façon formelle, l'ENI a été créée par le décret n° 83-185 du 24 mai 1983, comme étant le seul établissement universitaire professionnalisé au niveau national, destiné à former des techniciens et des ingénieurs de haut niveau, aptes à répondre aux besoins et aux exigences d'informatisation des entreprises, des sociétés et des organes implantés à Madagascar.

L'ENI a par conséquent pour mission de former des spécialistes informaticiens compétents et opérationnels de différents niveaux, notamment :

- en fournissant aux étudiants des connaissances de base en informatique ;
- en leur transmettant le savoir-faire requis, à travers la professionnalisation des formations dispensées et une meilleure adéquation entre les formations et les besoins évolutifs des sociétés et des entreprises ;

- en initiant les étudiants aux activités de recherche dans les différents domaines des Technologies de l'Information et de la Communication (TIC).

La filière de formation d'analystes programmeurs a été mise en place à l'école en 1983, puis gelée en 1996, tandis que la filière de formation d'ingénieurs a été ouverte en 1986.

Une formation de troisième cycle a été ouverte à l'école depuis l'année 2003-2004, grâce à la coopération académique et scientifique entre l'Université de Fianarantsoa, pour le compte de l'ENI, et l'Université Paul Sabatier de Toulouse (UPST). Cette filière avait pour objectif de former certains étudiants à la recherche dans les différents domaines de l'informatique, et notamment de préparer la relève des enseignants-chercheurs en poste.

Pendant l'année 2007-2008, la formation en vue de l'obtention du diplôme de Licence Professionnelle en Informatique a été mise en place à l'ENI avec deux parcours : Génie Logiciel et Bases de Données, d'une part, Administration des Systèmes et Réseaux, d'autre part. La mise en place de ces deux options devait répondre au besoin de basculement vers le système Licence – Master – Doctorat (LMD).

En vue de surmonter les difficultés liées à la limitation de l'effectif des étudiants accueillis, notamment à cause du manque d'infrastructures, un système de « formation hybride » a été mis en place à partir de l'année 2010. Il s'agit d'un système de formation semi-présentielle et à distance avec l'utilisation de la visioconférence. Ce système a été créé à Fianarantsoa ainsi qu'à l'Université de Toliara ; il est à l'origine du parcours Informatique Générale.

En 2023, la mention Intelligence Artificielle (IA) a été ouverte au sein de l'école pour répondre aux besoins des entreprises. La formation est destinée aux étudiants titulaires d'une licence (Bac + 3) en mathématiques, en statistiques ou en informatique. La mention IA comporte deux parcours : Gouvernance et Ingénierie de Données (GID) et Objets Connectés et Cybersécurité (OCC).

Depuis l'année universitaire 2024-2025, la mention Expertise Digitale est officiellement ouverte au grand public. Elle propose deux parcours distincts : Métiers du Digital (MDi), disponible aux niveaux Licence et Master, et Audit des Systèmes d'Information (ASI), proposé au niveau Master uniquement. Le système d'enseignement adopté pour cette mention est un système en ligne : les cours et ressources pédagogiques sont accessibles en permanence via la plateforme Moodle, et des regroupements en présentiel ou en ligne sont organisés périodiquement.

## 1.3. Organigramme institutionnel

Cet organigramme de l'école s'inspire des dispositions du décret n° 83-185 du 24 mai 1983. L'ENI est administrée par un conseil d'école et dirigée par un directeur nommé par décret adopté en Conseil des ministres. Le Collège des enseignants, qui regroupe tous les enseignants-chercheurs permanents de l'école, est chargé de résoudre les problèmes liés à l'organisation pédagogique des enseignements. Le Conseil scientifique propose les orientations pédagogiques et scientifiques de l'établissement, en tenant compte de l'évolution du marché du travail et de l'adéquation des formations dispensées avec les besoins des entreprises.

*Tableau 1. Organes de gouvernance de l'ENI*

| Organe | Rôle principal |
|---|---|
| Conseil d'école | Orientation générale, validation des décisions statutaires et budgétaires de l'établissement. |
| Directeur | Direction exécutive de l'école, nomination par décret en Conseil des ministres, représentation auprès des tutelles. |
| Conseil scientifique | Orientations pédagogiques et scientifiques, adéquation des formations avec le marché du travail. |
| Collège des enseignants | Organisation pédagogique des enseignements, délibérations et suivi des étudiants. |
| Responsables de mentions et de parcours | Pilotage des programmes, coordination des équipes pédagogiques et suivi des stages. |
| Services administratifs | Scolarité, ressources humaines, finances, informatique et logistique. |

## 1.4. Domaines de spécialisation

Les activités de formation et de recherche organisées à l'ENI portent notamment sur :

- Génie Logiciel et Bases de Données (GB) ;
- Administration des Systèmes et Réseaux (ASR) ;
- Informatique Générale (IG) ;
- Modélisation informatique et mathématique des systèmes complexes ;
- Intelligence Artificielle (IA).

Le système de formation pédagogique de l'école articule étroitement la théorie et la pratique, comme le montre le tableau intitulé « Organisation du système de formation ».

*Tableau 2. Organisation du système de formation*

| Formation théorique | Formation pratique |
|---|---|
| Enseignement théorique | Études de cas |
| Travaux dirigés | Travaux de réalisation |
| Travaux pratiques | Projets et projets tutorés |

| Formation théorique | Formation pratique |
|---|---|
| Conférences | Voyages d'études |
| — | Stage en entreprise |

## 1.5. Architecture des formations pédagogiques

Le recrutement des étudiants à l'ENI se fait uniquement par voie de concours d'envergure nationale en première année. Les offres de formation organisées par l'école ont été validées par la Commission Nationale d'Habilitation (CNH). Au sein de l'ENI, il existe plusieurs mentions et plusieurs parcours, récapitulés dans le tableau intitulé « Mentions et parcours au sein de l'ENI ».

*Tableau 3. Mentions et parcours au sein de l'ENI*

| Mention | Parcours |
|---|---|
| Informatique | Génie Logiciel et Bases de Données (GB) |
| Informatique | Administration des Systèmes et Réseaux (ASR) |
| Informatique | Informatique Générale (IG) |
| Intelligence Artificielle | Gouvernance et Ingénierie de Données (GID) |
| Intelligence Artificielle | Objets Connectés et Cybersécurité (OCC) |
| Expertise Digitale | Métiers du Digital (MDi) |
| Expertise Digitale | Audit des Systèmes d'Information (ASI) |

La licence peut avoir une vocation générale ou professionnelle. Le master peut avoir une vocation professionnelle ou de recherche. L'accès en première année de master est automatique pour les étudiants de l'école qui ont obtenu le diplôme de Licence Professionnelle.

*Tableau 4. Liste des formations existantes à l'ENI*

| Critère | Licence professionnelle | Master |
|---|---|---|
| Condition d'admission | Voie de concours | Voie de concours pour la mention IA |
| Condition d'accès | Baccalauréat série C, D ou Technique | Être titulaire d'une licence professionnelle |
| Durée de formation | 3 ans | 2 ans |
| Diplôme délivré | Diplôme de Licence Professionnelle | Master Professionnel ou Master Recherche |

Le Master Recherche permet à son titulaire de poursuivre directement des études en doctorat et de s'inscrire dans une école doctorale. Les étudiants diplômés de l'école sont plutôt bien accueillis dans les instituts universitaires étrangers (Canada, Suisse, France).

## 1.6. Relations de l'ENI avec les organismes externes

Les stages effectués chaque année par les étudiants mettent l'école en rapport permanent avec plus de 400 entreprises et organismes publics, semi-publics et privés, nationaux et internationaux. L'école dispose ainsi d'un réseau d'entreprises, de sociétés et d'organismes qui sont des partenaires pour l'accueil en stage de ses étudiants, et éventuellement pour le recrutement après l'obtention des diplômes.

Les compétences que l'école cherche à développer chez ses étudiants sont l'adaptabilité, le sens de la responsabilité, le travail en équipe, le goût de l'expérimentation et l'innovation. La vocation de l'ENI est de former des licenciés et des ingénieurs de niveau master avec des qualités scientifiques, techniques et humaines reconnues, capables d'évoluer professionnellement dans des secteurs d'activité variés intégrant l'informatique. Les stages en milieu professionnel permettent de favoriser une meilleure adéquation entre les formations de l'école et les besoins évolutifs du marché de l'emploi.

Parmi les partenaires de l'école, on peut citer : ACCENTURE Mauritius, Air Madagascar, Airtel, Agence Universitaire de la Francophonie (AUF), AXIAN, Banque Centrale, BIANCO, BlueLine, CNaPS, Bureau National de Gestion des Risques et des Catastrophes (BNGRC), Data Consulting, COLAS, Direction Générale des Douanes, FID, FTM, GNOSYS, INGENOSIA, INSTAT, JIRAMA, MADADEV, Ministère des Postes et Télécommunications, Orange, QMM Fort-Dauphin, Sénat, SOFTWELL, TELMA, WWF, entre autres.

## 1.7. Débouchés professionnels

Les formations proposées par l'école permettent aux diplômés d'être immédiatement opérationnels sur le marché du travail, avec la connaissance d'un métier complet lié à l'informatique et aux TIC. L'école apporte à ses étudiants un savoir-faire et un savoir-être qui les accompagnent tout au long de leur vie professionnelle. Les diplômés de licence et de master issus de l'ENI peuvent faire carrière dans différents secteurs, comme le résume le tableau intitulé « Débouchés éventuels des jeunes diplômés ».

*Tableau 5. Débouchés éventuels des jeunes diplômés*

| Mention | Débouchés professionnels |
|---|---|
| Informatique (GB) | Développeur logiciel ou full-stack, ingénieur en développement d'applications (desktop, web, mobile), architecte logiciel, développeur d'API, administrateur de bases de données, data engineer, analyste programmeur, analyste fonctionnel, chef de projet informatique, ingénieur QA, consultant fonctionnel. |

| Mention | Débouchés professionnels |
|---|---|
| Informatique (ASR) | Administrateur systèmes et réseaux, ingénieur systèmes et réseaux, administrateur cloud, ingénieur virtualisation, ingénieur cybersécurité, analyste SOC, technicien support, architecte réseaux, ingénieur DevOps, administrateur Kubernetes et Docker. |
| Intelligence Artificielle (GID) | Développeur IA, data scientist, consultant IA, data analyst, chef de produit, consultant en gouvernance des données. |
| Intelligence Artificielle (OCC) | Analyste et ingénieur en cybersécurité, spécialiste en réponse aux incidents, architecte de sécurité, ingénieur IoT, développeur IoT, auditeur en cybersécurité. |
| Expertise Digitale (MDi) | Spécialiste en marketing digital, gestionnaire de contenu web, analyste UX/UI, chef de projet digital, stratège en médias sociaux, consultant en transformation digitale. |
| Expertise Digitale (ASI) | Auditeur des systèmes d'information, consultant en sécurité des SI, analyste de risques IT, responsable de la conformité IT, expert en gouvernance IT. |

## 1.8. Ressources humaines

Les ressources humaines de l'école sont organisées ainsi :

- **Directeur de l'école** : Monsieur MAHATODY Thomas, Docteur HDR ;
- **Responsable de la mention « Informatique »** : Monsieur RABETAFIKA Louis Haja, Maître de conférences ;
- **Responsable de la mention « Intelligence Artificielle »** : Monsieur DIMBISOA William Germain, Maître de conférences ;
- **Responsable du parcours « Génie Logiciel et Bases de Données »** : Monsieur RALAIVAO Jean Christian ;
- **Responsable du parcours « Administration Systèmes et Réseaux »** : Monsieur SIAKA ;
- **Responsable du parcours « Informatique Générale »** : Monsieur GILANTE Gesazafy ;
- **Responsable du parcours « Gouvernance et Ingénierie de Données »** : Madame RATIANANTITRA Volatiana Marielle, Maître de conférences ;
- **Responsable du parcours « Objets Connectés et Cybersécurité »** : Monsieur RAZAFIMAHATRATRA Hajarisena, Maître de conférences.

L'ENI compte quinze enseignants permanents, dont un professeur titulaire, un professeur, un docteur HDR et huit maîtres de conférences, quatre assistants d'enseignement supérieur et de recherche, dix enseignants vacataires et quarante-et-un personnels administratifs.

# Chapitre 2. Présentation de la DGSR

## 2.1. Statut et missions

La Direction Générale de la Sécurité Routière (DGSR) est l'administration chargée de la mise en œuvre de la politique nationale de sécurité routière. Placée sous la tutelle du Ministère des Transports et de la Météorologie, elle exerce ses missions sur l'ensemble du territoire et s'appuie sur des structures déconcentrées, notamment les centres de réception technique (CRT) et le Centre National de Sécurité Routière (CENSERO).

Ses missions s'articulent autour de plusieurs axes :

- la **réglementation et le contrôle** des véhicules, par la réception technique et la vérification de leur conformité ;
- la **délivrance des titres** liés à la circulation et à la conduite, dans le respect des procédures réglementaires ;
- la **prévention et la sensibilisation** des usagers de la route, en partenariat avec les autres acteurs de la sécurité routière ;
- l'**exploitation des données** d'accidentalité et de contrôle, afin d'orienter les décisions publiques ;
- la **modernisation des services**, par la digitalisation des procédures et la fiabilisation des systèmes d'information.

La devise de l'institution, « Lahitokana ny Aina », rappelle que la sécurité routière est d'abord une affaire de preservation de la vie humaine. Cette exigence se traduit, au quotidien, par la rigueur attendue dans la tenue des dossiers, la traçabilité des opérations et la disponibilité des outils de travail.

## 2.2. Organisation administrative

La DGSR est structurée en directions et en services, eux-mêmes relayés par les structures territoriales. Le siège coordonne l'ensemble des activités, tandis que les CRT assurent la réception technique des véhicules dans les régions et que le CENSERO intervient sur les actions de sécurité routière. Le tableau intitulé « Directions et services de la DGSR » présente l'organisation telle qu'elle est enregistrée dans l'application GPI.

*Tableau 6. Directions et services de la DGSR*

| Direction | Code | Services rattachés | Attributions |
|---|---|---|---|
| Direction Générale | DG | Service Informatique, Secrétariat Central | Pilotage, coordination et représentation de l'établissement. |
| Direction Technique | DT | Service Planification et Études, Service Sécurité | Systèmes d'information, réception technique et immatriculation. |
| Direction des Opérations Routières | DOR | CRT Antananarivo, CRT Mahajanga, CENSERO Antananarivo | Centres de réception technique et opérations de sécurité routière. |
| Direction Administrative et Financière | DAF | Service Ressources Humaines, Service Finances et Budget | Ressources humaines, finances et logistique. |

Cette organisation explique la complexité du parc informatique à gérer : les équipements sont répartis entre le siège et les structures territoriales, les agents changent de service ou de direction, et les matériels circulent entre les sites. Une application centralisée devient dès lors indispensable pour conserver une vision fiable de l'affectation de chaque bien.

## 2.3. Organigramme fonctionnel

L'organigramme fonctionnel met en évidence les relations hiérarchiques et les responsabilités de chaque entité. La Direction Générale impulse la politique de l'établissement ; les directions techniques et opérationnelles mettent en œuvre les activités ; la Direction Administrative et Financière assure les moyens humains, financiers et matériels ; le Service Informatique, rattaché à la Direction Générale, intervient en support transversal.

*Tableau 7. Organigramme fonctionnel de la DGSR*

| Niveau | Entité | Responsabilité principale |
|---|---|---|
| 1 | Direction Générale | Définit les orientations, représente l'établissement, contrôle l'exécution des missions. |
| 2 | Direction Technique | Assure la réception technique, l'immatriculation et les systèmes d'information métier. |
| 2 | Direction des Opérations Routières | Pilote les CRT et le CENSERO, organise les opérations de contrôle et de prévention. |
| 2 | Direction Administrative et Financière | Gère les ressources humaines, le budget, les achats et la logistique. |
| 3 | Service Informatique | Administre le réseau, le parc informatique, les applications et la sécurité des systèmes. |
| 3 | Services et structures territoriales | Exécutent les activités opérationnelles et utilisent les outils mis à disposition. |

## 2.4. Valeurs et engagements

L'action de la DGSR s'appuie sur un ensemble de valeurs qui structurent également le fonctionnement de son service informatique :

- **Sécurité** : protéger les usagers, les agents et les données de l'établissement ;
- **Intégrité** : garantir la sincérité des informations et la traçabilité des opérations ;
- **Service public** : assurer la continuité et l'accessibilité des services rendus aux citoyens ;
- **Rigueur** : appliquer les procédures et contrôler la qualité des données ;
- **Innovation** : moderniser les outils et les pratiques pour gagner en efficacité.

## 2.5. Le Service Informatique

Le Service Informatique assure l'administration du réseau local, la gestion du parc, le support aux utilisateurs et la maintenance des applications métier. Il est l'interlocuteur direct du stage. Ses missions quotidiennes ont permis d'identifier précisément les besoins auxquels l'application GPI devait répondre :

- inventorier les équipements et connaître leur localisation exacte ;
- savoir à quel agent chaque matériel est affecté et depuis quelle date ;
- suivre les maintenances, les pannes et les demandes d'assistance ;
- conserver l'historique des achats, des fournisseurs et des garanties ;
- recenser les logiciels installés et surveiller les licences ;
- produire rapidement des états, des rapports et des documents officiels.

# Chapitre 3. Description du projet

## 3.1. Formulation

La gestion du parc informatique de la DGSR s'effectue encore en grande partie de manière manuelle : fichiers Excel tenus séparément par les services, registres de sortie de matériel, courriels pour signaler une panne, et connaissance informelle des affectations. Ce mode de fonctionnement entraîne des erreurs, des retards de traitement et une difficulté à suivre correctement l'état du parc. L'établissement a donc besoin d'une solution numérique permettant d'automatiser l'enregistrement des équipements, de centraliser les informations d'affectation et de permettre au responsable informatique de suivre facilement chaque matériel. Une application web intranet répond à ce besoin en offrant un système plus rapide, plus précis et plus simple à utiliser.

## 3.2. Objectif et besoins des utilisateurs

L'objectif du projet est de concevoir une application web de gestion du parc informatique permettant d'automatiser l'inventaire, de réduire les erreurs liées au suivi manuel et de faciliter l'accès aux données pour les responsables et l'administration. L'application est accessible depuis le réseau local de l'établissement, ce qui garantit la confidentialité des informations et un déploiement sans dépendance à Internet.

Pour répondre aux besoins des utilisateurs, l'application doit :

- permettre au service informatique d'enregistrer, de modifier et de consulter les équipements ;
- centraliser et sécuriser les données du parc (caractéristiques, prix, garanties, localisation) ;
- suivre les affectations et les restitutions de matériel auprès des agents ;
- organiser les maintenances et les tickets d'assistance, avec un historique ;
- gérer les achats, les fournisseurs et les licences logicielles ;
- offrir une consultation rapide de l'état du parc et des indicateurs clés ;
- générer des rapports et des exports PDF destinés à la hiérarchie ;
- distinguer les droits d'accès selon le rôle de chaque utilisateur.

## 3.3. Moyens nécessaires à la réalisation du projet

### *3.3.1. Moyens humains*

Le projet a mobilisé les compétences de plusieurs intervenants au sein de l'établissement : mon encadreur professionnel, responsable du Service Informatique, qui a défini les besoins et validé les choix fonctionnels ; les agents du service, qui ont participé aux tests et apporté des retours sur l'ergonomie ; les responsables administratifs, qui ont précisé les attentes en matière de rapports et de traçabilité. Mon encadreur pédagogique a assuré le suivi méthodologique et la relecture du rapport.

### *3.3.2. Moyens matériels*

Le développement a été réalisé sur un ordinateur portable équipé d'un processeur Intel Core i5, de 16 Go de mémoire vive et d'un disque SSD de 512 Go. Le serveur de test est un poste du réseau local de l'établissement, utilisé pour valider le fonctionnement en mode intranet. Les caractéristiques du poste de développement sont résumées dans le tableau intitulé « Caractéristiques du poste de développement ».

*Tableau 8. Caractéristiques du poste de développement*

| Élément | Caractéristique |
|---|---|
| Processeur | Intel Core i5 (4 cœurs, 2,4 GHz) |
| Mémoire vive | 16 Go DDR4 |
| Stockage | SSD NVMe 512 Go |
| Système d'exploitation | Windows 11 Professionnel 64 bits |
| Réseau | Connexion Ethernet et Wi-Fi au réseau local de l'établissement |
| Serveur de test | Poste fixe du Service Informatique, accessible par son adresse IP locale |

### *3.3.3. Moyens logiciels*

Les logiciels utilisés pour réaliser les différentes tâches du projet sont récapitulés dans le tableau intitulé « Logiciels utilisés et versions ». Les versions indiquées sont celles qui ont servi au développement et aux tests.

*Tableau 9. Logiciels utilisés et versions*

| Logiciel | Version | Usage |
|---|---|---|
| Visual Studio Code | 1.10x | Éditeur de code et terminal intégré |
| PHP | 8.4.15 | Langage serveur |
| Laravel | 13.32 | Framework PHP (API REST, ORM, sécurité) |
| Composer | 2.10.3 | Gestionnaire de dépendances PHP |
| MySQL | 8.4.7 | Système de gestion de base de données |
| phpMyAdmin | 5.2.3 | Administration de la base de données |

| Logiciel | Version | Usage |
|---|---|---|
| Node.js | 26.12.0 | Environnement JavaScript pour la compilation |
| npm | 11.20.0 | Gestionnaire de paquets JavaScript |
| React | 19.3 | Bibliothèque de l'interface utilisateur |
| Vite | 8.0 | Serveur de développement et compilation des assets |
| Tailwind CSS | 4.0 | Mise en forme de l'interface |
| Git | 2.56 | Gestion de versions |
| WampServer | 3.4 | Environnement serveur local (Apache, PHP, MySQL) |

Le serveur local WampServer fournit Apache, PHP et MySQL sur le poste de développement. Laravel est installé à l'aide de Composer, tandis que React, Vite et Tailwind CSS sont installés avec npm. L'ensemble est versionné avec Git, ce qui permet de revenir sur une modification et de suivre l'évolution du projet.

## 3.4. Résultats attendus

À la fin de la conception de la plateforme, il doit être facile pour chaque acteur de consulter les informations dont il a besoin et de réaliser ses tâches sans recourir à des fichiers dispersés. Les résultats attendus peuvent être présentés par profil d'utilisateur.

*Tableau 10. Résultats attendus par profil d'utilisateur*

| Profil | Résultats attendus |
|---|---|
| Administrateur | Accès complet au système : gestion des comptes, des rôles, des référentiels et du parc ; consultation des journaux d'activité ; suppression et restauration des éléments depuis la corbeille. |
| Gestionnaire de parc | Création et modification des équipements, des agents, des affectations, des achats et des fournisseurs ; production des rapports et des exports PDF. |
| Technicien | Traitement des tickets et des maintenances ; enregistrement des installations et désinstallations de logiciels ; commentaires sur les tickets. |
| Utilisateur | Consultation du parc et des fiches équipement ; déclaration d'un ticket d'assistance ; consultation de son profil. |
| Responsable / direction | Vue consolidée du parc : nombre d'équipements, valeur totale, répartition par direction et par statut, tickets ouverts, maintenances en cours, garanties proches de l'échéance. |

Au-delà de ces résultats fonctionnels, le projet doit permettre de réduire le temps de recherche d'une information, d'éviter les doubles saisies et de disposer d'un historique fiable des mouvements de matériel.

## 3.5. Chronogramme de travail

Le tableau intitulé « Diagramme de Gantt du travail pendant treize semaines » représente le chronogramme de travail durant lequel la plateforme a été conçue et développée. Le stage

s'est déroulé sur treize semaines, avec des phases qui se chevauchent partiellement afin de tester les modules au fur et à mesure de leur réalisation.

*Tableau 11. Diagramme de Gantt du travail pendant treize semaines*

| Activités | S1 | S2 | S3 | S4 | S5 | S6 | S7 | S8 | S9 | S10 | S11 | S12 | S13 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Phase 1. Analyse des besoins et conception du système | X | X | X | | | | | | | | | | |
| Phase 2. Installation de l'environnement et création de la base de données | | | X | X | X | | | | | | | | |
| Phase 3. Développement de l'API et des modules métier | | | | | X | X | X | X | X | | | | |
| Phase 4. Développement de l'interface React | | | | | | | X | X | X | X | | | |
| Phase 5. Tests, corrections et sécurisation | | | | | | | | | | X | X | X | |
| Phase 6. Rédaction du rapport et documentation | | | | | | | | | | | X | X | X |

Les phases 1 et 2 ont permis de cadrer le besoin et de préparer les outils. Les phases 3 et 4, menées en parallèle, ont constitué le cœur du développement : l'API a été construite module par module, puis consommée par l'interface. La phase 5 a servi à valider les règles de gestion et à corriger les anomalies détectées lors des tests. Enfin, la phase 6 a été consacrée à la rédaction du présent rapport et à la documentation du code.

# PARTIE II. ANALYSE ET CONCEPTION

Cette deuxième partie constitue le socle méthodologique du projet. Elle commence par un diagnostic de l'existant, puis présente les solutions envisagées et les choix technologiques retenus. Elle se poursuit par la modélisation du système d'information à l'aide de la méthode MERISE, depuis le dictionnaire des données jusqu'aux modèles conceptuel, logique, conceptuel des traitements et organisationnel des traitements.

# Chapitre 4. Analyse préalable

Cette phase a pour objectif d'établir un diagnostic complet du système actuel et de son environnement, afin de définir précisément les besoins et les contraintes du nouveau système.

## 4.1. Analyse de l'existant

### *4.1.1. Organisation actuelle*

Le Service Informatique de la DGSR assure aujourd'hui le suivi du parc à l'aide de plusieurs fichiers Excel, complétés par des registres papier et par des échanges de courriels. Chaque direction ou structure tient parfois son propre tableau, ce qui conduit à des informations différentes pour un même équipement.

- **Fonctionnalité principale** : saisie manuelle des équipements (désignation, numéro de série, prix, date d'achat), suivi sommaire des affectations et des pannes, calculs de base sur les montants.
- **Utilisateurs** : le Service Informatique et la Direction Administrative et Financière pour le suivi ; les agents n'ont pas d'interface directe pour consulter le matériel qui leur est confié.
- **Support** : fichiers partagés sur le réseau local, imprimés archivés dans des classeurs, appels téléphoniques pour les demandes d'intervention.

Ce mode de fonctionnement présente une limite structurelle : l'information n'est pas centralisée, elle n'est pas horodatée de manière fiable et elle ne permet pas de reconstituer l'historique des mouvements d'un matériel.

### *4.1.2. Inventaire des moyens matériels et logiciels*

L'infrastructure existante constitue une base solide pour le déploiement de la nouvelle solution.

**Matériel :**

- Postes utilisateurs : chaque agent du service administratif et technique dispose d'un ordinateur de bureau ou portable connecté au réseau local.
- Serveur local : l'établissement dispose d'un serveur pouvant héberger l'application et sa base de données.
- Réseau : un réseau local filaire et Wi-Fi couvre les bâtiments administratifs ; il est administré par le Service Informatique.

- Périphériques : imprimantes partagées, scanners, équipements réseau (commutateurs, bornes Wi-Fi), onduleurs.

**Logiciel :**

- Système d'exploitation : Windows 10 et Windows 11 sur les postes clients, Windows Server sur le serveur.
- Navigateurs web : Chrome, Firefox et Edge sont disponibles sur l'ensemble des postes.
- Suite bureautique : Microsoft Office, avec une forte dépendance à Excel pour le suivi actuel.
- Base de données : MySQL, déjà utilisée par plusieurs applications internes.

## 4.2. Critique de l'existant

L'analyse de l'existant fait apparaître des avantages réels, mais aussi des limites incompatibles avec les exigences de fiabilité et de traçabilité du service.

*Tableau 12. Avantages et inconvénients du système actuel*

| Avantages | Inconvénients et limites |
|---|---|
| Interface familière : les agents maîtrisent Excel et les tableurs. | Saisie manuelle, donc source d'erreurs et de doublons. |
| Consultation possible depuis plusieurs postes du réseau. | Aucun horodatage fiable des mouvements et des affectations. |
| Aucune formation nécessaire pour l'utilisation de base. | Informations dispersées entre plusieurs fichiers et services. |
| Flexibilité et adaptation rapide des tableaux. | Absence de règles de gestion : statuts incohérents, matériel affecté deux fois. |
| Maintenance minimale, sans compétence technique spécialisée. | Pas de gestion des rôles ni de traçabilité des consultations et modifications. |
| Coût logiciel nul. | Pas d'alertes sur les garanties, les licences ou les maintenances en retard. |
| — | Production de rapports lente et non standardisée ; pas d'export PDF officiel. |

## 4.3. Conception avant-projet

### *4.3.1. Proposition de solutions*

Après avoir analysé les problèmes rencontrés, la solution proposée doit répondre à plusieurs critères : être fiable, faciliter la gestion du parc, empêcher la modification non autorisée des données et rester simple à utiliser pour les agents. Trois pistes ont été examinées.

*Tableau 13. Comparaison des solutions envisagées*

| Solution envisagée | Avantages | Limites | Décision |
|---|---|---|---|
| Conserver Excel en l'améliorant (modèles, contrôles de saisie) | Coût nul, habitudes conservées | Pas de gestion des rôles, pas d'historique, pas d'accès concurrent fiable | Écartée |
| Acquérir un logiciel de gestion de parc du marché | Fonctionnalités complètes, support éditeur | Coût de licence, paramétrage lourd, dépendance à un prestataire | Écartée |
| Développer une application web intranet sur mesure | Adaptée aux processus de la DGSR, maîtrisée en interne, évolutive, sans licence | Nécessite un temps de développement et une maintenance applicative | Retenue |

La solution retenue est donc une application web intranet développée sur mesure. Elle sera accessible depuis un navigateur, sans installation sur les postes clients, et hébergée sur le serveur local de l'établissement. Cette approche garantit la maîtrise des données, la possibilité de faire évoluer l'outil et l'absence de coût de licence.

### *4.3.2. Méthodes de conception et outils*

Pour mener à bien l'étude, une méthode de conception appropriée a été mise en œuvre : la méthode MERISE (Méthode d'Étude et de Réalisation Informatique pour les Systèmes d'Entreprise). Elle permet de structurer les données et d'organiser les relations entre les différentes entités de la base de données. La méthode Agile a été examinée à titre comparatif, pour sa souplesse et sa capacité d'adaptation aux changements.

*Tableau 14. Comparaison méthodologique entre MERISE et Agile*

| Critère de comparaison | MERISE | AGILE |
|---|---|---|
| Philosophie de base | Approche systémique et séquentielle. | Approche itérative et adaptative. |
| Cycle de vie | « Planifier d'abord, exécuter après ». | « Expérimenter, apprendre, ajuster ». |
| Point de départ | Séquentiel en cascade : étapes strictes (conception, réalisation, validation). | Itératif en sprints : cycles courts de planification, développement et revue. |
| Gestion des risques | Préventive : les risques sont identifiés et traités pendant la conception. | Adaptative : les risques sont gérés au fur et à mesure des sprints. |
| Livrables | Dossier d'analyse, MCD, MLD, MCT, MOT, dossier technique. | Product Backlog, incréments fonctionnels, produit minimum viable. |

La méthode MERISE, développée en France dans les années 1970-1980, a été privilégiée en raison de sa forte orientation vers la modélisation des données et de sa structure rigoureuse, mieux adaptée à la conception d'un système d'information de gestion. Elle offre un cadre clair pour passer des besoins exprimés par le service aux tables de la base de données.

### *4.3.3. Choix du système de gestion de base de données*

La sélection d'un système de gestion de base de données est cruciale pour assurer l'efficacité et la performance de l'application. MySQL a été retenu pour sa simplicité

d'administration, sa compatibilité avec l'environnement PHP, ses performances sur les applications web de taille moyenne et sa présence dans l'infrastructure existante.

*Tableau 15. Tableau comparatif entre SGBDR*

| Critère | MySQL | PostgreSQL | Oracle Database |
|---|---|---|---|
| Type de SGBD | Relationnel | Relationnel / objet-relationnel | Relationnel |
| Licence | Open source (GPL) | Open source | Propriétaire (payant) |
| Coût | Gratuit | Gratuit | Très élevé |
| Complexité | Simple à moyen | Moyen à élevé | Très élevée |
| Performance | Très rapide pour les applications web | Très performant sur les requêtes complexes | Excellente sur de très gros volumes |
| Intégration PHP | Native (PDO, Eloquent) | Excellente | Nécessite des pilotes spécifiques |
| Administration | phpMyAdmin, outils simples | pgAdmin, configuration plus fine | Outils d'entreprise, expertise requise |

PostgreSQL et MySQL sont des SGBD open source largement utilisés dans les projets académiques et professionnels, tandis qu'Oracle est principalement destiné aux grandes entreprises nécessitant des performances et une sécurité très élevées, mais à un coût important. MySQL couvre largement les besoins du projet : volumétrie modérée, relations nombreuses, contraintes d'intégrité et transactions.

### *4.3.4. Choix du framework back-end*

Pour la partie serveur, le framework Laravel a été retenu. Laravel est un framework PHP open source reconnu pour sa robustesse, sa sécurité et sa rapidité de développement. Il adopte une architecture MVC (Modèle – Vue – Contrôleur) et fournit nativement les briques nécessaires au projet : routage, ORM Eloquent, validation, migrations, authentification par jetons et files d'attente.

*Tableau 16. Comparaison des frameworks back-end*

| Critère | Laravel | Symfony | Django (Python) |
|---|---|---|---|
| Langage | PHP 8.3+ | PHP 8.2+ | Python 3.12+ |
| Architecture | MVC | Composants / MVC | MVT (proche de MVC) |
| ORM | Eloquent (très expressif) | Doctrine | Django ORM |
| Authentification API | Sanctum intégré | À configurer (Security, JWT) | DRF + token/JWT |
| Migrations | Intégrées et simples | Doctrine Migrations | Intégrées |
| Courbe d'apprentissage | Modérée | Plus élevée | Modérée |
| Écosystème | Très riche, documentation abondante | Riche, orienté entreprise | Riche, orienté data/science |

Laravel a été préféré à Symfony pour sa simplicité de mise en route et à Django pour la cohérence avec l'environnement PHP déjà présent sur le serveur de l'établissement. Il permet de développer rapidement une API REST tout en conservant une structure de code claire et testable.

### *4.3.5. Choix de la technologie front-end*

L'interface utilisateur a été développée avec React, une bibliothèque JavaScript maintenue par Meta. React permet de construire des interfaces réactives à partir de composants réutilisables, ce qui convient parfaitement à une application comportant de nombreuses listes, formulaires et filtres.

*Tableau 17. Comparaison des technologies front-end*

| Critère | React | Vue.js | Angular |
|---|---|---|---|
| Nature | Bibliothèque UI | Framework progressif | Framework complet |
| Courbe d'apprentissage | Modérée (JSX, hooks) | Douce | Élevée (TypeScript, RxJS) |
| Écosystème | Très vaste | Vaste | Structuré, plus fermé |
| Performance | Très bonne (DOM virtuel) | Très bonne | Très bonne |
| Adéquation au projet | Élevée : composants, formulaires, tableaux | Élevée | Suffisante mais plus lourde |

React a été retenu pour la souplesse de son écosystème et la possibilité de composer l'interface à partir de composants métier (tableau d'équipements, fiche matériel, formulaire d'affectation, tableau de bord). Le routage est assuré par React Router et les appels réseau par Axios.

### *4.3.6. Environnement de développement et outils*

**Système de contrôle de version.** Git a été utilisé pour suivre les modifications du code et disposer d'un historique. Son modèle distribué permet de travailler localement, de créer des branches pour chaque fonctionnalité et de fusionner les évolutions. Subversion (SVN) et Mercurial ont été écartés : le premier dépend d'un serveur central, le second bénéficie d'une communauté plus restreinte.

**Éditeur de code.** Visual Studio Code a été choisi pour sa légèreté, ses extensions (PHP Intelephense, ESLint, Prettier, Tailwind CSS IntelliSense) et son terminal intégré, qui évite de multiplier les fenêtres pendant le développement.

**Compilation des assets.** Vite assure un rechargement rapide pendant le développement et produit des fichiers optimisés pour la production. Tailwind CSS fournit les classes utilitaires utilisées pour construire l'interface, tandis que Lucide React fournit les icônes.

**Serveur local.** WampServer regroupe Apache, PHP et MySQL sur le poste de développement. Il permet de reproduire fidèlement l'environnement du serveur de l'établissement et de tester l'application en mode intranet.

# Chapitre 5. Analyse conceptuelle

## 5.1. Présentation de la méthode MERISE

MERISE (Méthode d'Étude et de Réalisation Informatique pour les Systèmes d'Entreprise) est une méthode de conception de systèmes d'information développée en France dans les années 1970-1980. Elle propose une approche systémique et structurée, organisée autour de trois cycles et de trois niveaux d'abstraction.

**Les trois cycles MERISE :**

- **Cycle de vie** : enchaînement des étapes du projet, depuis l'étude préalable jusqu'à la mise en service et la maintenance (schéma directeur, étude préalable, étude détaillée, étude technique, production du logiciel, mise en service, maintenance).
- **Cycle de décision** : les étapes de validation par les décideurs (approbation et lancement, choix d'une solution, accord des utilisateurs, accord des réalisateurs, recette provisoire, conformité du système, recette définitive).
- **Cycle d'abstraction** : passage du monde réel au système informatique, en décrivant successivement les données, les traitements et les flux à trois niveaux : conceptuel, organisationnel, logique et physique.

*Tableau 18. Cycle d'abstraction MERISE*

| Système d'information | Niveau | Données | Traitements | Flux |
|---|---|---|---|---|
| Système d'information | Conceptuel | MCD : signification des informations, sans contrainte technique | MCT : activités du domaine sans préciser les ressources | MCF : relations entre le domaine et le reste du SI |
| Système d'information | Organisationnel | MOD : signification des informations avec contraintes | MOT : fonctionnement du domaine avec les ressources et leur organisation | MOF : relations entre les acteurs |
| Système d'information informatisé | Logique | MLD : description des données tenant compte des conditions de mémorisation | MLT : fonctionnement avec les ressources informatiques | MLF : relations entre les systèmes informatiques |
| Système d'information informatisé | Physique | MPD : description de la base dans la syntaxe du SGBD | MPT : architecture technique des programmes | MPF : supports techniques des flux |

Dans le cadre de ce projet, le niveau conceptuel a servi à comprendre le métier et à stabiliser le vocabulaire ; le niveau organisationnel a précisé qui fait quoi, où et avec quelles ressources ; le niveau logique a abouti au modèle relationnel implémenté dans MySQL.

## 5.2. Dictionnaire des données

Le dictionnaire des données répertorie toutes les données nécessaires à la construction du modèle conceptuel. Chaque rubrique est décrite par son type, sa taille, la possibilité qu'elle soit vide, son rôle (clé primaire ou étrangère) et une description métier. Plusieurs tableaux présentent, pour chaque entité, les rubriques classées par ordre alphabétique.

Le type **PRI** désigne une clé primaire, **FK** une clé étrangère et **UK** une contrainte d'unicité. La colonne « Null » indique si la donnée peut être vide.

### *a) Entité AGENT*

*Tableau 19. Dictionnaire des données — entité AGENT*

| Rubrique | Type | Taille | Null | Clé | Description |
|---|---|---|---|---|---|
| date_embauche | DATE | — | Oui | — | Date d'entrée dans l'établissement. |
| email | VARCHAR | 150 | Oui | — | Adresse électronique professionnelle. |
| fonction | VARCHAR | 100 | Oui | — | Poste occupé (technicien, chef de service, etc.). |
| id_agent | BIGINT | 20 | Non | PRI | Identifiant technique de l'agent (auto-incrémenté). |
| id_direction | BIGINT | 20 | Non | FK | Direction de rattachement. |
| id_service | BIGINT | 20 | Oui | FK | Service de rattachement. |
| matricule | VARCHAR | 30 | Non | UK | Matricule unique de l'agent (identifiant métier). |
| nom_naiss | VARCHAR | 100 | Non | — | Nom de naissance de l'agent. |
| prenom | VARCHAR | 100 | Non | — | Prénom de l'agent. |
| sexe | ENUM | M/F | Non | — | Sexe de l'agent. |
| statut | ENUM | — | Non | — | actif, inactif, congé ou mutation. |
| telephone | VARCHAR | 30 | Oui | — | Numéro de contact professionnel. |

### *b) Entité ÉQUIPEMENT*

*Tableau 20. Dictionnaire des données — entité ÉQUIPEMENT*

| Rubrique | Type | Taille | Null | Clé | Description |
|---|---|---|---|---|---|
| code_inventaire | VARCHAR | 50 | Non | UK | Code d'inventaire (ex. GPI-00001). |
| date_acquisition | DATE | — | Oui | — | Date d'achat ou d'entrée au parc. |
| date_mise_service | DATE | — | Oui | — | Date de mise en service. |
| description | TEXT | — | Oui | — | Description complémentaire. |
| designation | VARCHAR | 200 | Non | — | Désignation du matériel. |
| garantie_debut | DATE | — | Oui | — | Début de la garantie. |
| garantie_fin | DATE | — | Oui | — | Fin de la garantie (alertes à 60 jours). |
| id_categorie | BIGINT | 20 | Oui | FK | Catégorie (ordinateur, serveur, imprimante…). |

| Rubrique | Type | Taille | Null | Clé | Description |
|---|---|---|---|---|---|
| id_direction | BIGINT | 20 | Oui | FK | Direction d'affectation. |
| id_equipement | BIGINT | 20 | Non | PRI | Identifiant technique de l'équipement. |
| id_etat | BIGINT | 20 | Oui | FK | État physique constaté. |
| id_fournisseur | BIGINT | 20 | Oui | FK | Fournisseur du matériel. |
| id_localisation | BIGINT | 20 | Oui | FK | Localisation physique (site, bâtiment, bureau). |
| id_marque | BIGINT | 20 | Oui | FK | Marque du matériel. |
| id_modele | BIGINT | 20 | Oui | FK | Modèle du matériel. |
| id_service | BIGINT | 20 | Oui | FK | Service d'affectation. |
| numero_serie | VARCHAR | 100 | Oui | UK | Numéro de série constructeur. |
| prix_acquisition | DECIMAL | 14,2 | Oui | — | Prix d'achat en ariary. |
| statut | ENUM | — | Non | — | en_service, en_stock, en_maintenance, reforme, perdu, reserve. |

### *c) Entité AFFECTATION*

*Tableau 21. Dictionnaire des données — entité AFFECTATION*

| Rubrique | Type | Taille | Null | Clé | Description |
|---|---|---|---|---|---|
| commentaire | TEXT | — | Oui | — | Observations complémentaires. |
| date_affectation | DATE | — | Non | — | Date de remise du matériel. |
| date_fin | DATE | — | Oui | — | Date de restitution (nulle si en cours). |
| id_affectation | BIGINT | 20 | Non | PRI | Identifiant de l'affectation. |
| id_agent | BIGINT | 20 | Non | FK | Agent bénéficiaire. |
| id_equipement | BIGINT | 20 | Non | FK | Équipement concerné. |
| motif | TEXT | — | Oui | — | Motif de l'affectation. |

### *d) Entité TICKET*

*Tableau 22. Dictionnaire des données — entité TICKET*

| Rubrique | Type | Taille | Null | Clé | Description |
|---|---|---|---|---|---|
| date_cloture | DATE | — | Oui | — | Date de clôture ou de résolution. |
| date_creation | DATE | — | Oui | — | Date d'ouverture du ticket. |
| description | TEXT | — | Oui | — | Description détaillée du problème. |
| id_agent | BIGINT | 20 | Oui | FK | Agent demandeur. |
| id_equipement | BIGINT | 20 | Oui | FK | Équipement concerné. |
| id_ticket | BIGINT | 20 | Non | PRI | Identifiant du ticket. |
| priorite | ENUM | — | Non | — | basse, moyenne, haute ou critique. |
| reference_ticket | VARCHAR | 30 | Non | UK | Référence générée (TCK-année-numéro). |
| statut | ENUM | — | Non | — | ouvert, en_cours, resolu, ferme. |
| titre | VARCHAR | 200 | Non | — | Objet de la demande. |

### e) Entité MAINTENANCE

*Tableau 23. Dictionnaire des données — entité MAINTENANCE*

| Rubrique | Type | Taille | Null | Clé | Description |
|---|---|---|---|---|---|
| date_debut | DATE | — | Oui | — | Date de début de l'intervention. |
| date_demande | DATE | — | Oui | — | Date de la demande d'intervention. |
| date_fin | DATE | — | Oui | — | Date de fin de l'intervention. |
| description_probleme | TEXT | — | Oui | — | Panne ou symptômes constatés. |
| diagnostic | TEXT | — | Oui | — | Diagnostic et actions réalisées. |
| id_agent | BIGINT | 20 | Oui | FK | Agent à l'origine de la demande. |
| id_equipement | BIGINT | 20 | Non | FK | Équipement en maintenance. |
| id_maintenance | BIGINT | 20 | Non | PRI | Identifiant de l'intervention. |
| id_type_maintenance | BIGINT | 20 | Non | FK | Type d'intervention (préventive, curative…). |
| statut | ENUM | — | Non | — | demandee, planifiee, en_cours, terminee, annulee. |
| technicien | VARCHAR | 150 | Oui | — | Technicien en charge du dossier. |

### f) Entité ACHAT

*Tableau 24. Dictionnaire des données — entité ACHAT*

| Rubrique | Type | Taille | Null | Clé | Description |
|---|---|---|---|---|---|
| bon_commande | VARCHAR | 50 | Oui | — | Référence du bon de commande. |
| date_achat | DATE | — | Oui | — | Date de la commande ou de l'achat. |
| id_achat | BIGINT | 20 | Non | PRI | Identifiant de l'achat. |
| id_fournisseur | BIGINT | 20 | Oui | FK | Fournisseur retenu. |
| montant_total | DECIMAL | 14,2 | Oui | — | Montant total de l'achat. |
| numero_facture | VARCHAR | 50 | Oui | — | Numéro de facture fournisseur. |
| reference_achat | VARCHAR | 50 | Oui | UK | Référence interne du marché ou de la commande. |
| statut | ENUM | — | Non | — | commande, livre_partiel, livre, annule. |

### g) Entité LOGICIEL

*Tableau 25. Dictionnaire des données — entité LOGICIEL*

| Rubrique | Type | Taille | Null | Clé | Description |
|---|---|---|---|---|---|
| date_expiration | DATE | — | Oui | — | Date d'expiration de la licence. |
| editeur | VARCHAR | 150 | Oui | — | Éditeur du logiciel. |
| id_logiciel | BIGINT | 20 | Non | PRI | Identifiant du logiciel. |
| nom | VARCHAR | 150 | Non | — | Nom commercial du logiciel. |
| reference_licence | VARCHAR | 50 | Non | UK | Référence unique de la licence. |
| version | VARCHAR | 50 | Oui | — | Version de référence. |

## 5.3. Règles de gestion

Les règles de gestion (RG) sont les contraintes métier qui régissent le fonctionnement du système. Elles garantissent l'intégrité, la cohérence et la conformité aux processus de la DGSR. Elles ont été traduites dans le code, dans les validations et dans les contraintes de la base de données.

*Tableau 26. Règles de gestion du système*

| Code | Règle de gestion | Traduction technique |
|---|---|---|
| RG01 | Un équipement possède un code d'inventaire unique et un numéro de série unique s'il est renseigné. | Contraintes UNIQUE sur code_inventaire et numero_serie. |
| RG02 | Un agent possède un matricule unique et immuable. | Contrainte UNIQUE sur matricule. |
| RG03 | Un équipement ne peut être affecté qu'à un seul agent à la fois. | Contrôle applicatif à la création d'une affectation ; date_fin nulle pour l'affectation en cours. |
| RG04 | Une affectation terminée ne peut plus être modifiée, seule la restitution est autorisée. | Route dédiée /affectations/{id}/restitution ; mise à jour de date_fin. |
| RG05 | Un ticket possède une référence unique de la forme TCK-année-numéro. | Génération dans une transaction, contrainte UNIQUE sur reference_ticket. |
| RG06 | Un ticket clôturé ou résolu renseigne automatiquement sa date de clôture. | Contrôle dans TicketController lors du changement de statut. |
| RG07 | Un équipement en maintenance ne peut pas être affecté à un nouvel agent. | Contrôle de statut avant création d'affectation. |
| RG08 | La suppression d'un équipement ou d'un agent est logique (corbeille) et réversible. | SoftDeletes sur equipements et agents ; restauration depuis la corbeille. |
| RG09 | Un utilisateur ne peut accéder qu'aux fonctionnalités autorisées par son rôle. | Middleware role:… et policies côté API. |
| RG10 | Un agent ne peut consulter que les données du parc, pas les modifier. | Rôle utilisateur en lecture seule ; routes d'écriture protégées. |
| RG11 | Toute opération sensible est tracée dans le journal d'activité. | Table journal_activites et notifications associées. |
| RG12 | Une licence expirée ne peut plus être associée à une nouvelle installation. | Contrôle du statut de la licence avant installation. |

Ces règles ont été définies avec le Service Informatique lors des entretiens de cadrage. Elles constituent la référence pour les tests de validation présentés au chapitre 7.

## 5.4. Diagramme de flux

Les modèles de flux représentent ce qui doit être étudié dans le cadre du projet, à partir de l'analyse des flux échangés. Le diagramme de flux est une représentation graphique des acteurs et des informations qui circulent entre eux.

- **Le domaine** : découpage de l'organisation correspondant à une finalité majeure ; ici, la gestion du parc informatique.

- **L'acteur** : unité active intervenant dans le fonctionnement du système opérant (service, agent, application).
- **Le flux** : échange d'informations entre acteurs, matérialisé par un document, une requête ou une notification.

### *5.4.1. Acteurs du système*

*Tableau 27. Acteurs du système*

| Acteur | Rôle | Responsabilités |
|---|---|---|
| Administrateur | Gestionnaire du système | Gestion des comptes et des rôles, configuration, supervision, journal d'activité. |
| Gestionnaire de parc | Responsable du parc | Création et mise à jour des équipements, affectations, achats, fournisseurs et référentiels. |
| Technicien | Intervenant technique | Traitement des tickets et des maintenances, installations logicielles, diagnostics. |
| Agent | Utilisateur final | Consultation du matériel affecté, déclaration d'un ticket, consultation de son profil. |
| Système GPI | Application Laravel / React | Traitement, validation, stockage et restitution des données. |
| Base de données MySQL | Persistance | Stockage des données, contraintes d'intégrité, historique. |
| Service de messagerie / intranet | Diffusion | Mise à disposition de l'application sur le réseau local. |

### *5.4.2. Détail des flux d'informations*

**Flux 1 — Gestion du parc.** L'administrateur ou le gestionnaire saisit les caractéristiques d'un équipement (désignation, marque, modèle, numéro de série, prix, garantie). Le système valide les champs obligatoires et l'unicité du code d'inventaire, puis enregistre l'équipement et renvoie une confirmation. En cas d'erreur, un message explicite indique le champ concerné.

**Flux 2 — Authentification et accès.** L'agent saisit son identifiant et son mot de passe. Le système vérifie les informations, génère un jeton d'accès et renvoie le profil avec les rôles associés. L'interface adapte ensuite les menus et les actions autorisées. Une tentative échouée est limitée par un mécanisme de limitation de débit (six tentatives par minute).

**Flux 3 — Affectation et restitution.** Le gestionnaire sélectionne un équipement disponible et un agent, puis enregistre l'affectation avec une date et un motif. Le système vérifie que l'équipement n'est pas déjà affecté et met à jour son statut. À la restitution, la date de fin est renseignée et l'équipement redevient disponible.

**Flux 4 — Assistance et maintenance.** L'agent déclare un ticket (titre, description, priorité). Le technicien le prend en charge, change son statut, ajoute des commentaires et, si nécessaire, crée une maintenance liée. Chaque changement de statut déclenche une notification pour le demandeur et pour les intervenants.

**Gestion des erreurs.** Les erreurs de saisie sont signalées à l'utilisateur par un message clair et un champ mis en évidence. Les erreurs système sont journalisées et une alerte est envoyée à l'administrateur. Les exports PDF en échec sont relancés après vérification des données source.

## 5.5. Modélisation des données

### *5.5.1. Modèle Conceptuel des Données (MCD)*

Le MCD est une représentation graphique de haut niveau qui permet de comprendre simplement comment les différents éléments du système sont liés entre eux. Faisant partie de la boîte à outils MERISE, il décrit les données utilisées par le système d'information et leurs relations. Les informations sont représentées logiquement à l'aide d'un ensemble de règles et de diagrammes codifiés :

- les **entités** (un rectangle = un objet) ;
- les **propriétés** (la liste des données de l'entité) ;
- les **relations** qui expliquent comment les entités sont reliées entre elles (losanges) ;
- les **cardinalités** (les chiffres portés sur chaque patte de la relation).

La construction du MCD suit quatre étapes : obtenir les règles de gestion du besoin à informatiser, recenser les données dans un dictionnaire, comprendre les relations et les dépendances entre les données, puis construire le modèle. Le MCD du projet GPI est présenté à la figure 1. Il regroupe les entités du parc, de l'organisation, des achats, de la maintenance, de l'inventaire et de la traçabilité.

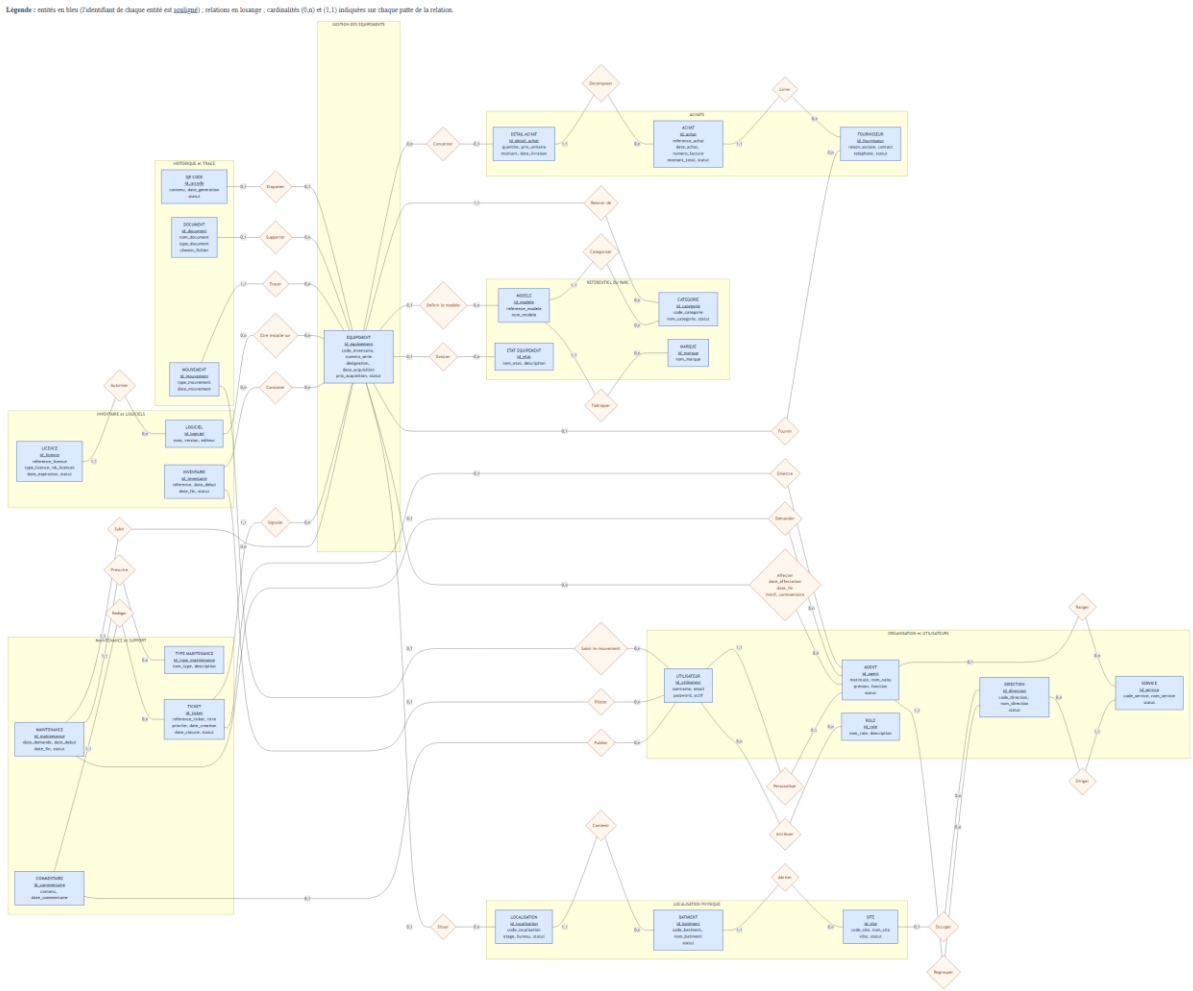


*Figure 1. Modèle Conceptuel des Données (MCD) du système GPI*

**Interprétation des principales relations :**

- Une **direction** regroupe zéro ou plusieurs **services** ; un service relève d'une et une seule direction.
- Un **service** emploie zéro ou plusieurs **agents** ; un agent est rattaché à une direction et, facultativement, à un service.

- Un **agent** peut recevoir plusieurs **affectations** ; une affectation concerne un seul agent et un seul équipement.
- Un **équipement** peut faire l'objet de plusieurs **affectations**, **maintenances**, **tickets** et **mouvements** ; chaque opération référence un seul équipement.
- Un **ticket** peut contenir plusieurs **commentaires** ; chaque commentaire est rédigé par un utilisateur.
- Un **achat** comporte plusieurs **lignes de détail** ; chaque ligne peut être rattachée à un équipement.
- Un **logiciel** possède plusieurs **licences** et peut être installé sur plusieurs équipements.
- Un **inventaire** comporte plusieurs **lignes de constat**, chacune portant sur un équipement.
- Un **équipement** possède au plus un **QR code** et peut être associé à plusieurs **documents**.

### *5.5.2. Modèle Logique des Données (MLD)*

Le MLD est une représentation textuelle de la base de données à informatiser, consécutive au travail d'analyse du MCD et précédant le modèle physique. Le passage du MCD au MLD applique six règles :

- **Règle 1** : une entité devient une relation (table) ; ses propriétés deviennent des attributs et son identifiant devient la clé primaire.
- **Règle 2** : pour une relation binaire de cardinalités (X,1) – (X,n), la clé primaire de la table côté (X,n) devient une clé étrangère dans la table côté (X,1).
- **Règle 3** : pour une relation binaire (X,n) – (X,n), une table supplémentaire est créée, avec une clé primaire composée des identifiants des deux entités.
- **Règle 4** : pour une relation n-aire, une table est créée avec la concaténation des identifiants des entités participantes.
- **Règle 5** : pour une association réflexive, la clé primaire se dédouble et devient clé étrangère dans la même table, ou une table de liaison est créée selon les cardinalités.
- **Règle 6** : pour une relation binaire (0,1) – (1,1), la clé primaire de la table côté (0,1) devient une clé étrangère dans la table côté (1,1).

Le MLD obtenu comporte les relations principales (les clés primaires sont soulignées dans le modèle graphique et les clés étrangères sont précédées du symbole #) :

- **DIRECTION** (id_direction, code_direction, nom_direction, description, telephone, email, statut)
- **SERVICE** (id_service, code_service, nom_service, description, statut, #id_direction)
- **AGENT** (id_agent, matricule, nom_naiss, prenom, sexe, fonction, telephone, email, date_embauche, statut, #id_direction, #id_service)
- **UTILISATEUR** (id_utilisateur, username, email, password, actif, dernier_connexion, #id_agent)
- **ROLE** (id_role, nom_role, description) — **ROLE_UTILISATEUR** (#id_utilisateur, #id_role)
- **SITE** (id_site, code_site, nom_site, ville, statut) — **BATIMENT** (id_batiment, code_batiment, nom_batiment, statut, #id_site) — **LOCALISATION** (id_localisation, code_localisation, etage, bureau, statut, #id_batiment)
- **CATEGORIE** (id_categorie, code_categorie, nom_categorie, statut) — **MARQUE** (id_marque, nom_marque) — **MODELE** (id_modele, reference_modele, nom_modele, #id_categorie, #id_marque)
- **ETAT_EQUIPEMENT** (id_etat, nom_etat, description)
- **EQUIPEMENT** (id_equipement, code_inventaire, numero_serie, designation, date_acquisition, date_mise_service, prix_acquisition, garantie_debut, garantie_fin, statut, #id_modele, #id_categorie, #id_marque, #id_direction, #id_service, #id_fournisseur, #id_localisation, #id_etat, #id_detail_achat)
- **FOURNISSEUR** (id_fournisseur, raison_sociale, contact, telephone, email, adresse, rcs, statut)
- **ACHAT** (id_achat, reference_achat, date_achat, numero_facture, bon_commande, montant_total, statut, #id_fournisseur)
- **DETAIL_ACHAT** (id_detail_achat, quantite, prix_unitaire, montant, date_livraison, #id_achat, #id_equipement)
- **AFFECTATION** (id_affectation, date_affectation, date_fin, motif, commentaire, #id_equipement, #id_agent)
- **TYPE_MAINTENANCE** (id_type_maintenance, nom_type, description) — **MAINTENANCE** (id_maintenance, date_demande, date_debut, date_fin,

description_probleme, diagnostic, technicien, statut, #id_equipement, #id_type_maintenance, #id_agent)

- **TICKET** (id_ticket, reference_ticket, titre, description, priorite, date_creation, date_cloture, statut, #id_equipement, #id_agent) — **TICKET_COMMENTAIRE** (id_commentaire, contenu, date_commentaire, #id_ticket, #id_utilisateur)
- **INVENTAIRE** (id_inventaire, reference, date_debut, date_fin, description, statut, #id_utilisateur) — **INVENTAIRE_DETAIL** (id_detail, presence, date_constat, observation, statut, #id_inventaire, #id_equipement)
- **LOGICIEL** (id_logiciel, nom, version, editeur, description) — **LICENCE** (id_licence, reference_licence, type_licence, date_acquisition, date_expiration, numero_serie, statut, #id_logiciel) — **INSTALLATION_LOGICIEL** (id_installation, date_installation, version_installee, licence, statut, #id_logiciel, #id_equipement)
- **MOUVEMENT** (id_mouvement, type_mouvement, date_mouvement, #id_equipement, #ancienne_localisation, #nouvelle_localisation, #id_utilisateur) — **MOUVEMENT_HISTORIQUE** (id_historique, #id_mouvement, #id_utilisateur)
- **DOCUMENT** (id_document, nom_document, type_document, chemin_fichier, #id_equipement, #id_utilisateur) — **QR_CODE** (id_qrcode, contenu, date_generation, statut, #id_equipement)
- **NOTIFICATION** (id_notification, titre, message, lu, #id_utilisateur) — **JOURNAL_ACTIVITE** (id_journal, action, entite, #id_utilisateur)

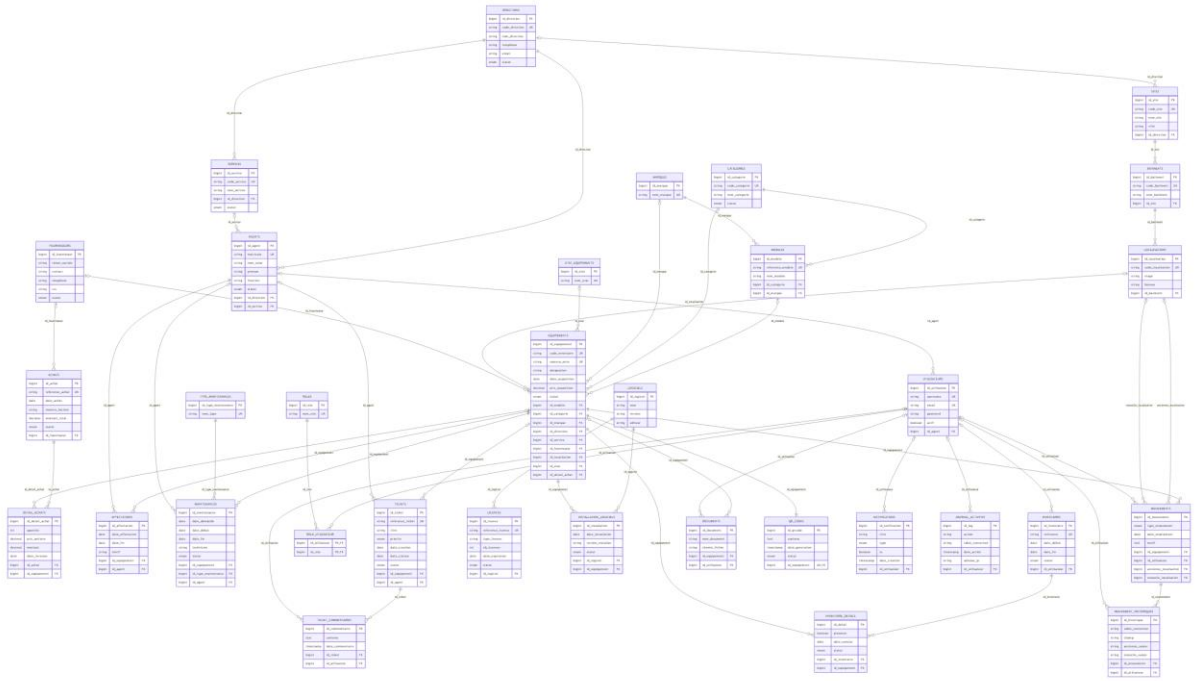


*Figure 2. Modèle Logique des Données (MLD) du système GPI*

### *5.5.3. Modèle Conceptuel des Traitements (MCT)*

Le MCT permet de traiter la dynamique du système d'information, c'est-à-dire les opérations réalisées en fonction d'événements. Il représente de façon schématique l'activité du système sans faire référence à des choix organisationnels ou à des moyens d'exécution : il définit ce qui doit être fait, mais ne dit ni quand, ni comment, ni où. Deux MCT ont été produits pour couvrir les processus principaux du projet.

Le premier décrit le cycle **tickets, affectations et inventaire** : déclaration d'un ticket par un agent, qualification et affectation à un technicien, création d'une maintenance si nécessaire, clôture et mise à jour de l'historique. Le second décrit la **restitution et la réforme** d'un matériel : constat de l'état, décision de restitution ou de réforme, mise à jour du statut et génération du mouvement.

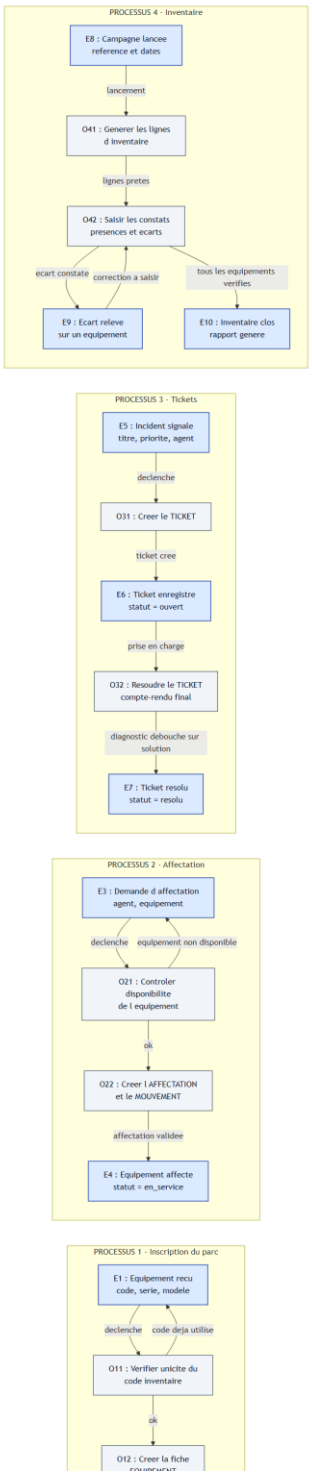



*Figure 3. Modèle Conceptuel des Traitements (MCT) — tickets, affectations et inventaire*

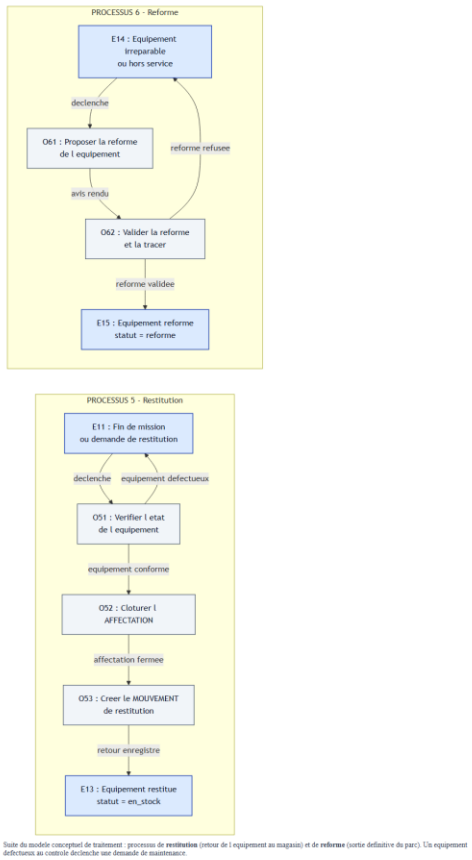



*Figure 4. Modèle Conceptuel des Traitements (MCT) — restitution et réforme*

### *5.5.4. Modèle Organisationnel des Traitements (MOT)*

Le modèle organisationnel des traitements décrit les propriétés des traitements non traitées par le modèle conceptuel, à savoir le temps, les ressources et le lieu. Il représente le modèle conceptuel des traitements dans un tableau dont les colonnes précisent la durée, le lieu, les responsables et les ressources nécessaires à chaque action.

La première étape consiste à découper les opérations en procédures fonctionnelles, c'est-à-dire une succession de traitements déclenchée par un événement. Le tableau associe ensuite le traitement, son déroulement, l'acteur, le lieu et les ressources du poste de travail.

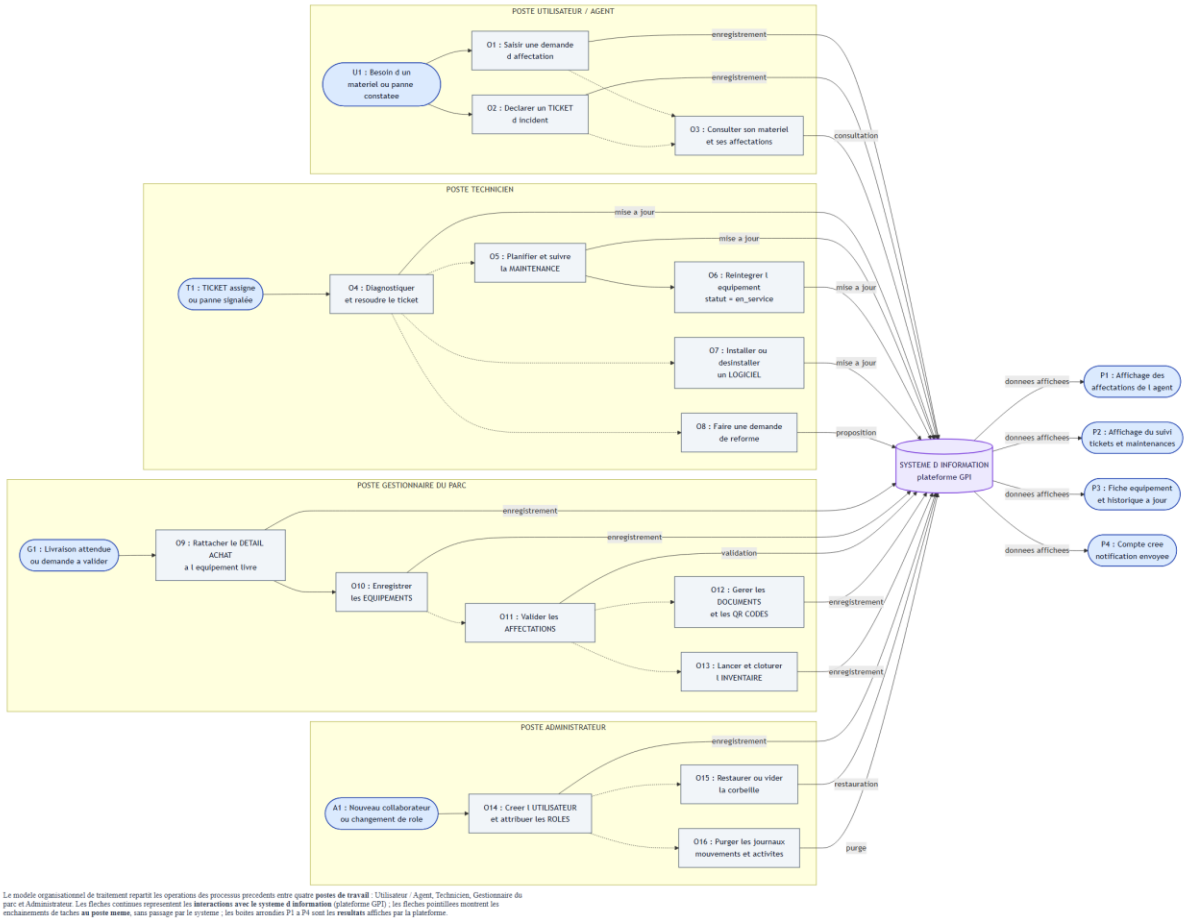



*Figure 5. Modèle Organisationnel des Traitements (MOT) du système GPI*

*Tableau 28. Traitements, acteurs et ressources du système*

| Traitement | Acteur | Lieu | Ressources | Durée |
|---|---|---|---|---|
| Enregistrer un équipement | Gestionnaire de parc | Bureau du service informatique | Poste connecté, navigateur, fiche de réception | 5 min |
| Affecter un matériel | Gestionnaire de parc | Bureau du service informatique | Poste connecté, bon de remise | 3 min |
| Déclarer un ticket | Agent | Tout poste du réseau | Navigateur, identifiants | 2 min |
| Traiter un ticket | Technicien | Atelier technique | Poste connecté, outils de diagnostic | 15 à 60 min |
| Restituer un matériel | Gestionnaire de parc | Bureau du service informatique | Poste connecté, fiche de restitution | 5 min |
| Lancer un inventaire | Administrateur | Bureau du service informatique | Poste connecté, liste du parc | Variable |
| Éditer un rapport PDF | Gestionnaire / direction | Bureau ou salle de réunion | Navigateur, imprimante | 2 min |

# PARTIE III. RÉALISATION

Cette troisième partie présente la mise en œuvre concrète du projet : installation et configuration des outils, architecture de l'application, création de la base de données, développement des modules, présentation des interfaces, tests et perspectives d'évolution.

# Chapitre 6. Mise en place de l'environnement de développement

Le choix et la configuration des outils conditionnent la qualité du développement, la reproductibilité de l'installation et la facilité de maintenance. Ce chapitre décrit l'environnement mis en place sur le poste de développement, puis l'architecture générale de l'application.

## 6.1. Installation et configuration des outils

### *a) Visual Studio Code*

Visual Studio Code est un éditeur de code gratuit et open source développé par Microsoft. Il prend en charge de nombreux langages et dispose d'un terminal intégré, ce qui évite de multiplier les fenêtres pendant le développement. L'installation se fait à partir du site officiel, puis les extensions complémentaires ont été ajoutées :

- **PHP Intelephense** : complétion, navigation et détection d'erreurs en PHP ;
- **Laravel Blade Snippets** et **Laravel Extra Intellisense** : assistance sur les vues et les routes ;
- **ESLint** et **Prettier** : qualité et formatage du code JavaScript et JSX ;
- **Tailwind CSS IntelliSense** : complétion des classes utilitaires ;
- **GitLens** : historique et annotations Git directement dans l'éditeur.

La vérification de l'installation se fait depuis un terminal :

```
code -v
1.107.0
```

### *b) WampServer, Apache et PHP*

WampServer regroupe Apache, PHP et MySQL sur le poste de développement. Il reproduit fidèlement l'environnement du serveur de l'établissement et permet de tester l'application en mode intranet. La version installée est WampServer 3.4, avec PHP 8.4.15 et MySQL 8.4.7.

Les extensions PHP nécessaires à Laravel et à la génération des PDF ont été activées dans le fichier php.ini :

- **pdo_mysql** : accès à MySQL par PDO ;
- **mbstring** et **iconv** : gestion des chaînes de caractères accentuées ;

- **openssl** : chiffrement et génération des jetons ;
- **fileinfo** et **gd** : traitement des fichiers et des images, notamment pour les QR codes ;
- **zip** : gestion des archives et de l'installation des dépendances.

```
php -v
PHP 8.4.15 (cli) (built: Nov 18 2025 18:38:40) (ZTS Visual C++ 2022 x64)
```

### *c) Composer et Laravel*

Composer est le gestionnaire de dépendances de PHP. Il permet d'installer Laravel et ses bibliothèques, puis de régénérer l'autoloader. La version utilisée est Composer 2.10.3.

```
composer -V
Composer version 2.10.3 2026-08-27 13:34:23

composer create-project laravel/laravel gpi
cd gpi
composer install
```

Après la création du projet, le fichier d'environnement est initialisé et la clé d'application générée. Cette clé sert au chiffrement des sessions et des jetons.

```
copy .env.example .env
php artisan key:generate

php artisan serve
INFO  Server running on [http://127.0.0.1:8000].
```

### *d) Node.js, npm, Vite et Tailwind CSS*

Node.js fournit l'environnement d'exécution JavaScript nécessaire à la compilation des assets. npm installe React, Vite, Tailwind CSS et les autres dépendances front-end. Node.js 26.12.0 et npm 11.20.0 ont été utilisés.

```
node -v
v26.12.0
npm -v
11.20.0

npm install
npm run dev      # serveur de développement Vite (HMR)
npm run build    # compilation pour la production
```

En développement, Vite sert les fichiers JavaScript et CSS avec rechargement à chaud. En production, la commande npm run build génère les fichiers compilés dans public/build, avec une empreinte dans le nom des fichiers pour éviter les problèmes de cache.

### *e) MySQL et phpMyAdmin*

MySQL assure le stockage des données. La base a été créée avec l'interclassement utf8mb4_unicode_ci, afin de gérer correctement les caractères accentués et les emojis. phpMyAdmin 5.2.3 a servi à administrer la base, à importer le script gpi.sql et à vérifier les contraintes.

```
CREATE DATABASE gpi
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
```

La connexion est configurée dans le fichier .env, qui n'est jamais versionné :

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=gpi
DB_USERNAME=root
DB_PASSWORD=
```

Les tables sont créées par le mécanisme de migrations de Laravel, puis peuplées par les seeders pour disposer d'un jeu de données de démonstration.

```
php artisan migrate --seed
INFO  Running migrations... DONE
INFO  Seeding database... DONE
```

### *f) Git*

Git a été utilisé pour suivre les modifications du code et conserver un historique des évolutions. Après l'installation, l'identité de l'auteur est configurée, puis le dépôt est initialisé.

```
git --version
git version 2.56.0.windows.2

git config --global user.name "etudiant"
git config --global user.email "etudiant@eni.mg"
git init
```

Les commandes utilisées durant le développement sont :

```
git status
git add .
git commit -m "Ajout du module des affectations"
git log --oneline
```

## 6.2. Architecture de l'application

L'application repose sur une architecture trois tiers, qui sépare clairement la présentation, la logique métier et le stockage des données. Cette séparation facilite la maintenance, l'évolution et la sécurisation de l'ensemble.

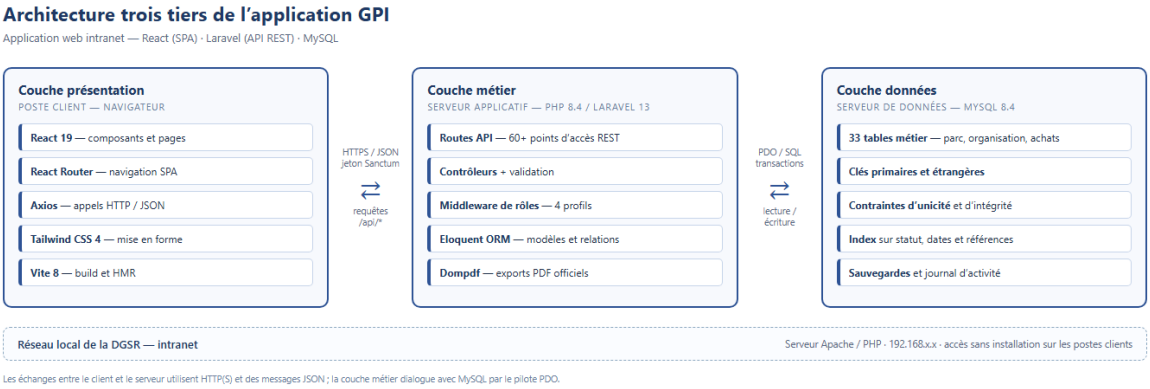



*Figure 6. Architecture trois tiers de l'application GPI*

### *6.2.1. Couche présentation*

La couche présentation est une application mono-page (SPA) développée avec React 19. Elle est servie par Laravel à travers une vue unique et charge les données depuis l'API. Le routage côté client est assuré par React Router : les adresses /equipements, /tickets, /rapports, etc. affichent les pages correspondantes sans rechargement complet. Axios centralise les appels HTTP et ajoute automatiquement le jeton d'authentification à chaque requête.

L'interface est découpée en composants réutilisables : barre latérale de navigation, en-tête, tableaux, formulaires, fenêtres de confirmation, notifications et composants de tableau de bord. Tailwind CSS fournit les classes de mise en forme, tandis que Lucide React fournit les icônes.

### *6.2.2. Couche métier*

La couche métier est assurée par Laravel 13. Elle expose une API REST sous le préfixe /api, authentifiée par Laravel Sanctum. Les contrôleurs valident les données, appliquent les règles de gestion, écrivent dans la base via l'ORM Eloquent et renvoient des réponses JSON. Un middleware de rôles protège les routes d'écriture.

Les principaux modules sont :

- **Organisation** : directions, services, agents, utilisateurs et rôles ;
- **Localisation** : sites, bâtiments et localisations précises ;

- **Parc** : catégories, marques, modèles, états et équipements ;
- **Gestion** : fournisseurs, achats et affectations ;
- **Maintenance** : types d'intervention, maintenances, tickets et commentaires ;
- **Inventaire** : campagnes d'inventaire et lignes de constat ;
- **Logiciels** : catalogue, licences et installations ;
- **Traçabilité** : mouvements, historique, documents, QR codes, notifications et journal d'activité ;
- **Rapports** : synthèses, exports CSV et PDF officiels.

### *6.2.3. Couche données*

La couche données repose sur MySQL 8.4. La base comporte trente-trois tables métier, liées par des clés étrangères et protégées par des contraintes d'unicité et d'intégrité. Les index portent sur les colonnes les plus sollicitées : statut des équipements, dates d'affectation, références de tickets et clés étrangères. Les sauvegardes sont réalisées par export SQL et par la fonctionnalité de sauvegarde du serveur.

### *6.2.4. Communication entre les couches*

Le client et le serveur échangent en JSON par HTTP. Les requêtes d'écriture sont protégées par le jeton Sanctum et par la vérification du rôle. Les réponses d'erreur respectent les codes HTTP usuels : 200 pour un succès, 201 pour une création, 401 pour une absence d'authentification, 403 pour un accès refusé, 404 pour une ressource introuvable et 422 pour une erreur de validation. Ce fonctionnement permet à l'interface d'afficher des messages précis sans interpréter des chaînes de caractères.

## 6.3. Structure du projet

Le projet suit la structure standard de Laravel, complétée par une organisation claire des pages React. La structure simplifiée est :

```
gpi/
├── app/
│   ├── Http/
│   │   ├── Controllers/      # 16 contrôleurs métier
│   │   └── Middleware/       # EnsureUserHasRole
│   ├── Models/               # 33 modèles Eloquent
│   └── Support/              # notifications métier
├── database/
│   ├── migrations/           # 17 migrations
│   └── seeders/              # jeu de données de démonstration
├── docs/merise/              # MCD, MLD, MCT, MOT
├── public/                   # point d'entrée et assets compilés
├── resources/
│   ├── css/app.css
│   ├── js/
│   │   ├── app.jsx           # routes et layout
│   │   ├── api.js            # client Axios
│   │   ├── auth.jsx          # contexte d'authentification
│   │   ├── components/       # composants réutilisables
│   │   └── pages/            # pages métier
│   └── views/app.blade.php
├── routes/
│   ├── api.php               # API REST
│   └── web.php               # SPA (toutes les routes non-API)
├── storage/                  # logs, cache, fichiers générés
├── .env
├── composer.json
└── package.json
```

## 6.4. Déploiement en mode intranet

L'application est destinée à être utilisée sur le réseau local de la DGSR. Le serveur de l'établissement héberge Apache, PHP et MySQL ; les postes clients n'ont besoin que d'un navigateur. Le déploiement suit plusieurs étapes :

- récupération du code et installation des dépendances avec Composer ;
- configuration du fichier .env avec les paramètres du serveur (base, URL, environnement de production) ;
- génération de la clé d'application, exécution des migrations et des seeders ;
- compilation des assets avec npm run build ;
- configuration du virtual host Apache sur le dossier public, afin que les fichiers sensibles ne soient pas accessibles ;
- mise en place des droits d'écriture sur storage et bootstrap/cache ;
- sauvegarde de la base et mise en place d'une procédure de restauration.

```
<VirtualHost *:80>
    ServerName gpi.dgsr.local
    DocumentRoot "C:/wamp64/www/gpi/public"

    <Directory "C:/wamp64/www/gpi/public">
        AllowOverride All
        Require all granted
    </Directory>
</VirtualHost>
```

Une fois déployée, l'application est accessible depuis les postes du réseau à l'adresse http://gpi.dgsr.local ou, à défaut, à l'adresse IP du serveur. Aucune installation n'est requise sur les postes clients.

# Chapitre 7. Développement de l'application

## 7.1. Création de la base de données

La création de la base de données constitue une étape essentielle : elle permet de stocker et d'organiser de manière structurée l'ensemble des informations du parc. Le modèle relationnel issu du MLD a été traduit en tables MySQL, puis enrichi de contraintes et d'index.

**Choix du SGBD.** MySQL a été retenu pour sa simplicité d'administration, sa compatibilité native avec PHP, ses performances sur les applications web de taille moyenne et sa présence dans l'infrastructure existante. Il respecte les contraintes d'intégrité, gère les transactions et s'intègre parfaitement à l'ORM Eloquent.

**Mise en place.** Les tables sont créées par les migrations de Laravel, ce qui garantit la reproductibilité de l'installation et le suivi des évolutions du schéma. Le script gpi.sql permet également de restaurer la base complète, données de démonstration comprises.

Les principales tables métier sont décrites dans le tableau intitulé « Principales tables de la base GPI ».

*Tableau 29. Principales tables de la base GPI*

| Table | Rôle |
|---|---|
| directions / services | Structure organisationnelle de l'établissement. |
| agents / utilisateurs / roles / role_utilisateur | Personnel, comptes d'accès et gestion des rôles. |
| sites / batiments / localisations | Hiérarchie géographique du parc. |
| categories / marques / modeles / etat_equipements | Référentiels de classification et d'état. |
| equipements | Inventaire central : code, série, prix, garantie, statut, rattachements. |
| fournisseurs / achats / detail_achats | Achats, marchés et lignes de livraison. |
| affectations | Historique des remises et restitutions de matériel. |
| type_maintenances / maintenances | Interventions préventives et curatives. |
| tickets / ticket_commentaires | Demandes d'assistance et échanges associés. |
| inventaires / inventaire_details | Campagnes de vérification physique du parc. |
| logiciels / licences / installation_logiciels | Catalogue logiciel, licences et installations. |
| mouvements / mouvement_historiques | Traçabilité des entrées, sorties, transferts et réformes. |
| documents / qr_codes | Pièces jointes et étiquetage des équipements. |
| notifications / journal_activites | Alertes utilisateurs et journal des opérations. |

**Respect des règles d'intégrité.** Plusieurs contraintes ont été mises en place : unicité du code d'inventaire, du numéro de série, du matricule et de la référence de ticket ; clés étrangères avec restrictions ou mises à zéro selon le cas ; suppression logique des équipements et des

agents pour préserver l'historique ; contrôle applicatif des doublons d'affectation. Ces règles garantissent la cohérence et la fiabilité du système.

## 7.2. Codage de l'application

Le codage transforme les modèles conceptuels en une application fonctionnelle. Le développement a été réalisé avec Laravel pour le back-end et React pour l'interface, selon une organisation par module et en appliquant les bonnes pratiques du framework.

### *7.2.1. Les modèles Eloquent*

Chaque table possède un modèle Eloquent qui décrit ses champs, ses conversions de types et ses relations. Les clés primaires personnalisées sont déclarées explicitement, car Laravel utilise par défaut la colonne id. Les dates sont converties automatiquement en objets Carbon, ce qui simplifie les calculs et le formatage.

```
class Equipement extends Model
{
    use SoftDeletes;

    protected $primaryKey = "id_equipement";

    protected $fillable = [
        "code_inventaire", "numero_serie", "designation",
        "date_acquisition", "prix_acquisition", "garantie_fin",
        "statut", "id_modele", "id_categorie", "id_localisation",
    ];

    protected function casts(): array
    {
        return [
            "date_acquisition" => "date",
            "prix_acquisition" => "decimal:2",
        ];
    }

    public function categorie(): BelongsTo
    {
        return $this->belongsTo(Categorie::class, "id_categorie");
    }
}
```

Les relations sont définies dans les deux sens lorsque c'est utile : un équipement appartient à une catégorie, une marque, un modèle et une localisation ; il possède plusieurs affectations, maintenances, tickets, installations et mouvements. Ces relations permettent d'éviter les requêtes manuelles et de charger les données liées en une seule opération.

### *7.2.2. Les contrôleurs et l'API REST*

Les contrôleurs exposent les opérations CRUD et appliquent la validation. Une classe de base, CrudController, mutualise la logique commune (liste, création, mise à jour,

suppression, journalisation) ; les contrôleurs métier héritent de cette classe et redéfinissent les règles et les relations à charger. Ce choix réduit la duplication et homogénéise les réponses de l'API.

Les routes sont regroupées par domaine et protégées par le middleware de rôles. Le tableau intitulé « Principaux points d'accès de l'API » présente les principaux points d'accès.

*Tableau 30. Principaux points d'accès de l'API*

| Méthode | Point d'accès | Description | Rôle requis |
|---|---|---|---|
| POST | /api/login | Authentification et génération du jeton | Public |
| GET | /api/me | Profil de l'utilisateur connecté | Authentifié |
| GET | /api/dashboard | Indicateurs du tableau de bord | Authentifié |
| GET | /api/equipements | Liste filtrée et paginée des équipements | Authentifié |
| POST | /api/equipements | Création d'un équipement | Admin, gestionnaire |
| GET | /api/equipements/{id} | Fiche détaillée d'un équipement | Authentifié |
| PUT | /api/equipements/{id} | Modification d'un équipement | Admin, gestionnaire |
| DELETE | /api/equipements/{id} | Mise à la corbeille | Admin, gestionnaire |
| GET | /api/agents | Répertoire des agents | Authentifié |
| POST | /api/affectations | Affectation d'un matériel | Admin, gestionnaire |
| POST | /api/affectations/{id}/restitution | Restitution d'un matériel | Admin, gestionnaire |
| GET | /api/tickets | Liste des tickets | Authentifié |
| POST | /api/tickets | Déclaration d'un ticket | Authentifié |
| POST | /api/tickets/{id}/commentaires | Ajout d'un commentaire | Authentifié |
| GET | /api/maintenances | Suivi des interventions | Authentifié |
| GET | /api/inventaires | Campagnes d'inventaire | Authentifié |
| GET | /api/rapports/synthese | Synthèse du parc | Authentifié |
| GET | /api/rapports/pdf/synthese | Export PDF de synthèse | Authentifié |
| GET | /api/corbeille | Éléments supprimés | Authentifié |
| PUT | /api/corbeille/{section}/{id}/restaurer | Restauration d'un élément | Admin, gestionnaire |

### *7.2.3. Authentification, rôles et sécurité*

L'authentification repose sur Laravel Sanctum. À la connexion, le serveur vérifie l'identifiant et le mot de passe, puis génère un jeton personnel stocké côté client. Ce jeton est

transmis dans l'en-tête Authorization de chaque requête. La route de connexion est limitée à six tentatives par minute pour freiner les attaques par force brute.

Quatre rôles ont été définis, avec des permissions croissantes :

*Tableau 31. Rôles et permissions de l'application*

| Rôle | Permissions principales |
|---|---|
| admin | Accès complet : comptes, rôles, référentiels, parc, rapports, corbeille, journal d'activité. |
| gestionnaire_parc | Écriture sur le parc, les agents, les affectations, les achats, les fournisseurs et les référentiels. |
| technicien | Tickets, maintenances, installations et désinstallations de logiciels ; lecture du parc. |
| utilisateur | Lecture du parc et des fiches, déclaration de tickets, consultation de son profil. |

Le contrôle d'accès est assuré par le middleware EnsureUserHasRole, qui vérifie que l'utilisateur possède au moins un des rôles exigés par la route. En cas de refus, une réponse JSON 403 explicite est renvoyée, ce qui permet à l'interface d'afficher un message compréhensible.

```
class EnsureUserHasRole
{
    public function handle(Request $request, Closure $next, string ...$roles): Response
    {
        $user = $request->user();

        $autorise = $user !== null && $user->roles->contains(
            fn ($role) => in_array($role->nom_role, $roles, true)
        );

        if (! $autorise) {
            return response()->json([
                "message" => "Action non autorisée pour votre rôle.",
            ], 403);
        }

        return $next($request);
    }
}
```

D'autres mesures renforcent la sécurité : validation systématique des entrées, échappement des sorties par React, requêtes paramétrées par Eloquent, protection CSRF pour les formulaires web, mots de passe hachés avec bcrypt, jetons révocables et journalisation des actions sensibles.

### *7.2.4. Application des règles de gestion*

Les règles définies lors de la conception sont intégrées directement dans le code. La génération de la référence d'un ticket illustre cette approche : plutôt qu'un comptage susceptible

de produire des doublons en cas de créations simultanées, la référence est dérivée de l'identifiant auto-incrémenté, à l'intérieur d'une transaction.

```
$ticket = DB::transaction(function () use ($data) {
    $data["reference_ticket"] = "TCK-".date("Y")."-TMP-".uniqid();
    $ticket = Ticket::create($data);

    $ticket->update([
        "reference_ticket" => "TCK-".date("Y")."-"
            .str_pad((string) $ticket->id_ticket, 3, "0", STR_PAD_LEFT),
    ]);

    return $ticket;
});
```

D'autres contrôles sont appliqués : un équipement déjà affecté ne peut pas être affecté une seconde fois ; un ticket résolu ou fermé renseigne automatiquement sa date de clôture ; un équipement en maintenance ne peut pas être remis à un agent ; une licence expirée ne peut plus être associée à une nouvelle installation. Les messages d'erreur sont renvoyés au format JSON avec le détail des champs concernés.

### *7.2.5. Exports PDF et documents officiels*

Les rapports peuvent être consultés à l'écran, exportés en CSV ou générés en PDF. La bibliothèque Dompdf produit des documents comportant l'en-tête officiel de l'établissement, la date d'édition, la pagination et un emplacement réservé au visa. Quatre types d'exports sont disponibles : la synthèse du parc, l'historique des mouvements, le journal d'activité et la fiche détaillée d'un équipement. La mise en page est définie dans des vues Blade dédiées, ce qui permet de faire évoluer le modèle de document sans modifier la logique métier.

### *7.2.6. L'interface React*

L'interface est organisée en pages métier et en composants partagés. Le fichier app.jsx déclare les routes protégées : un utilisateur non authentifié est redirigé vers /connexion, tandis qu'un utilisateur connecté accède au tableau de bord et aux modules. Le contexte d'authentification conserve le profil et les rôles, ce qui permet de masquer les actions non autorisées.

Les principales pages sont : tableau de bord, équipements, fiche équipement, affectations, tickets, maintenances, inventaires, achats, fournisseurs, logiciels, agents, directions, services, sites, bâtiments, localisations, catégories, marques, modèles, rapports, corbeille et paramètres. Les tableaux intègrent la recherche, les filtres, la pagination et les actions contextuelles. Les formulaires affichent les erreurs de validation renvoyées par l'API, champ par champ.

L'application propose également un mode clair et un mode sombre, ainsi qu'un basculement français/anglais pour les libellés de l'interface. Ces fonctionnalités améliorent le confort d'utilisation sans alourdir le code métier.

### *7.2.7. Tests et validation*

Des tests manuels ont été effectués à chaque étape du développement, complétés par des vérifications automatisées sur les points critiques. Ils ont porté sur le bon fonctionnement des fonctionnalités, le respect des règles de gestion et la cohérence entre la base de données, l'API et l'interface. Le tableau intitulé « Cas de tests et résultats » présente les principaux cas testés.

*Tableau 32. Cas de tests et résultats*

| Cas de test | Scénario | Résultat attendu | État |
|---|---|---|---|
| Authentification valide | Connexion avec admin@dgsr.mg et le mot de passe correct | Jeton généré, redirection vers le tableau de bord | Conforme |
| Authentification invalide | Mot de passe erroné répété | Message d'erreur, limitation après six tentatives | Conforme |
| Accès non autorisé | Technicien tentant de supprimer un équipement | Réponse 403 et message explicite | Conforme |
| Création d'équipement | Saisie complète avec code unique | Équipement créé, visible dans la liste | Conforme |
| Doublon de code | Création avec un code d'inventaire existant | Erreur 422, aucun doublon en base | Conforme |
| Affectation | Attribution d'un matériel disponible à un agent | Affectation enregistrée, statut mis à jour | Conforme |
| Double affectation | Nouvelle affectation d'un matériel déjà affecté | Refus et message d'erreur | Conforme |
| Restitution | Retour d'un matériel affecté | Date de fin renseignée, matériel disponible | Conforme |
| Ticket | Déclaration puis changement de statut | Référence TCK générée, date de clôture renseignée | Conforme |
| Export PDF | Génération de la synthèse du parc | PDF téléchargé avec en-tête et pagination | Conforme |
| Corbeille | Suppression puis restauration d'un agent | Élément masqué puis restauré avec ses liens | Conforme |
| Inventaire | Constat de présence et écart | Ligne enregistrée, statut de la campagne mis à jour | Conforme |

### *7.2.8. Difficultés rencontrées et solutions*

Le développement n'a pas été linéaire. Plusieurs difficultés ont dû être résolues, parfois au prix d'une refonte partielle de certaines parties du code.

*Tableau 33. Difficultés rencontrées et solutions*

| Difficulté | Cause | Solution apportée |
|---|---|---|
| Conflit de routage entre l'API et la SPA | Toutes les URL non-API renvoyaient la page unique, y compris les appels API mal formés. | Route attrape-tout limitée par une expression régulière excluant le préfixe api. |
| Sessions et jetons en développement | Le front et le back n'utilisaient pas le même domaine. | Configuration CORS et utilisation de jetons Sanctum plutôt que des cookies de session. |
| Accents dans les PDF | Polices par défaut de Dompdf incomplètes. | Choix d'une police prenant en charge l'UTF-8 et déclaration explicite dans la vue PDF. |
| Suppression et historique | La suppression physique cassait les liens avec les affectations et les mouvements. | Suppression logique (SoftDeletes) et corbeille avec restauration. |
| Performances des listes | Chargement des relations une par une (problème N+1). | Utilisation de with() pour charger les relations en une requête et pagination des résultats. |
| Doublons de référence de ticket | Comptage des tickets existants en cas de créations simultanées. | Référence dérivée de l'identifiant, générée dans une transaction. |

## 7.3. Présentation de l'application

Ce chapitre présente l'application à travers ses interfaces et ses fonctionnalités. Il met en évidence le fonctionnement global du système, depuis l'authentification jusqu'à la gestion du parc, des affectations, des maintenances et des rapports.

### *7.3.1. Page d'authentification*

La première interaction de l'utilisateur avec l'application se fait à travers la page d'authentification. Cette interface permet de se connecter de manière sécurisée avec un identifiant et un mot de passe. Elle rappelle le cadre institutionnel, la devise de l'établissement et le caractère réservé de l'accès.

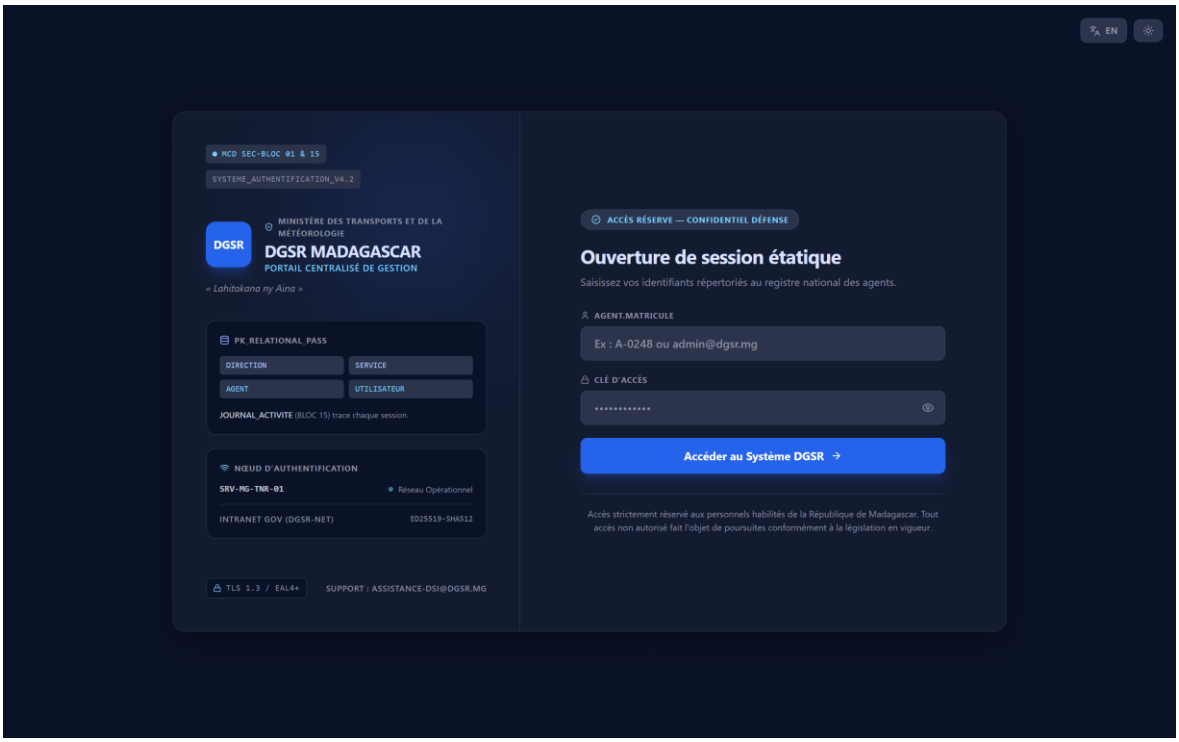



*Figure 7. Page d'authentification de l'application GPI*

Une fois les informations vérifiées, le serveur renvoie un jeton et le profil de l'utilisateur avec ses rôles. L'interface adapte alors les menus et les actions disponibles : un agent ne voit pas les boutons de suppression ou de création réservés au gestionnaire.

### *7.3.2. Tableau de bord*

Après une connexion réussie, l'utilisateur est redirigé vers le tableau de bord. Celui-ci présente une vue d'ensemble du parc : nombre de matériels répertoriés, tickets ouverts, maintenances en cours et affectations actives. Des graphiques complètent cette synthèse : répartition par statut, par catégorie et par direction, valeur totale du parc et garanties proches de l'échéance.

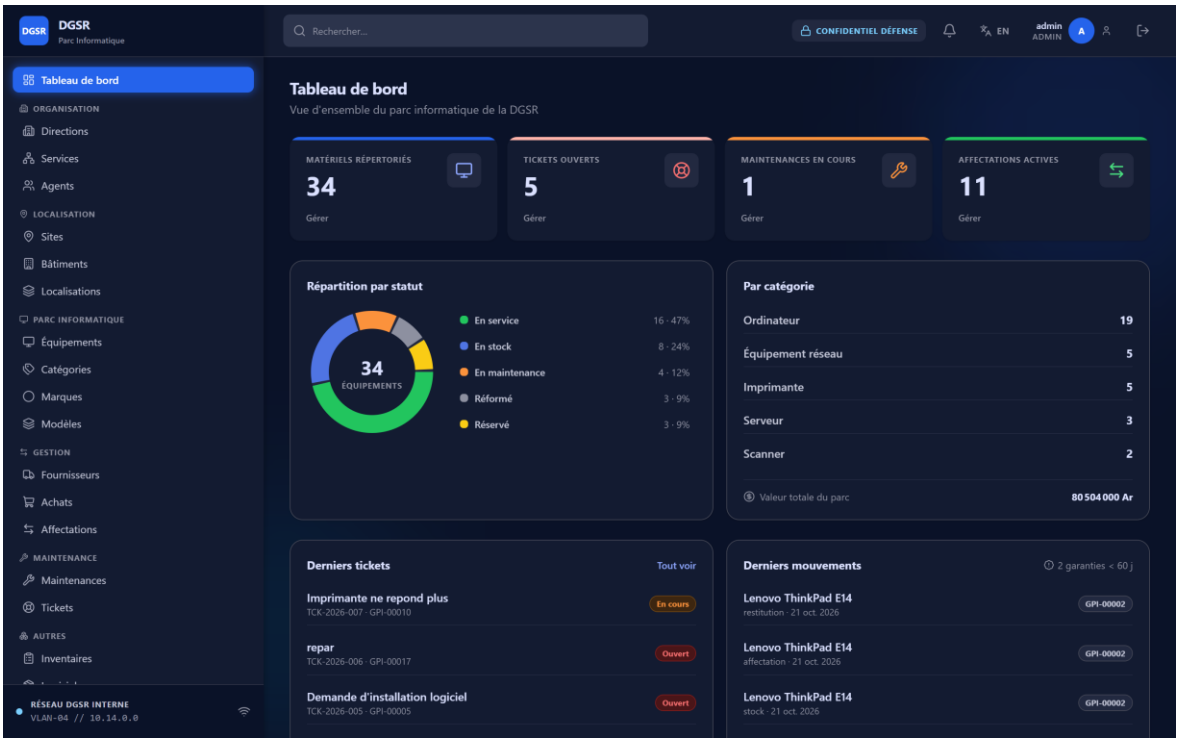



*Figure 8. Tableau de bord : indicateurs et répartition du parc*

Les indicateurs sont calculés côté serveur à partir des données réelles : au moment de la rédaction, le parc compte trente-quatre équipements, seize en service, huit en stock, quatre en maintenance, trois réformés et trois réservés, pour une valeur totale de 80 504 000 ariary. Onze affectations sont en cours et cinq tickets restent ouverts.

### *7.3.3. Gestion des équipements*

La page des équipements affiche la liste complète du parc avec le code d'inventaire, la désignation, la catégorie, la localisation et le statut. La recherche porte sur le code, le numéro de série ou la désignation ; des filtres permettent de restreindre la liste par statut et par catégorie. Chaque ligne propose les actions de consultation, de modification et de suppression.

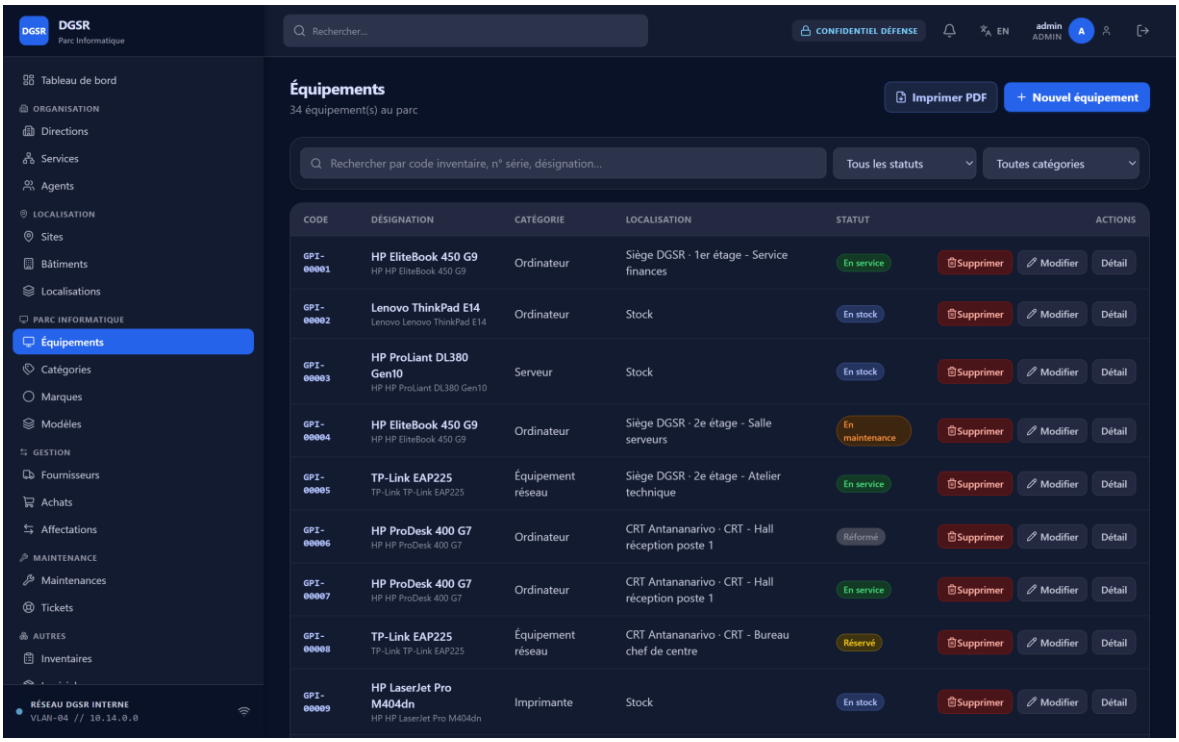



*Figure 9. Liste des équipements avec recherche et filtres*

### *7.3.4. Fiche détaillée d'un équipement*

La fiche d'un équipement regroupe l'ensemble de son cycle de vie : caractéristiques, direction et service, fournisseur, localisation, date d'acquisition, prix, garantie, affectations, maintenances et tickets liés, logiciels installés, mouvements et documents. Cette vue unique évite de naviguer entre plusieurs modules pour reconstituer l'histoire d'un matériel.

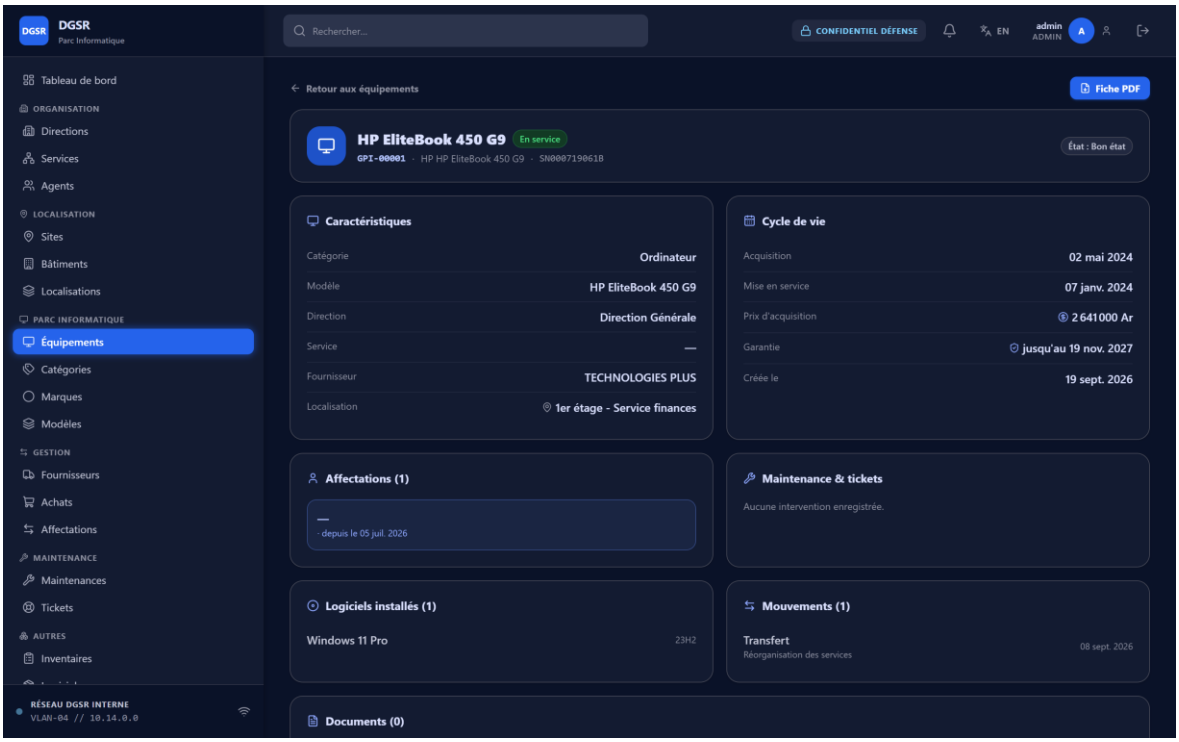



*Figure 10. Fiche détaillée d'un équipement et historique*

### *7.3.5. Affectations et restitutions*

La page des affectations distingue les affectations en cours et les affectations terminées. Elle indique l'agent, la fonction, le code d'inventaire, la désignation et la date de remise. Le bouton « Restituer » clôt l'affectation en renseignant la date de fin et en rendant le matériel disponible.

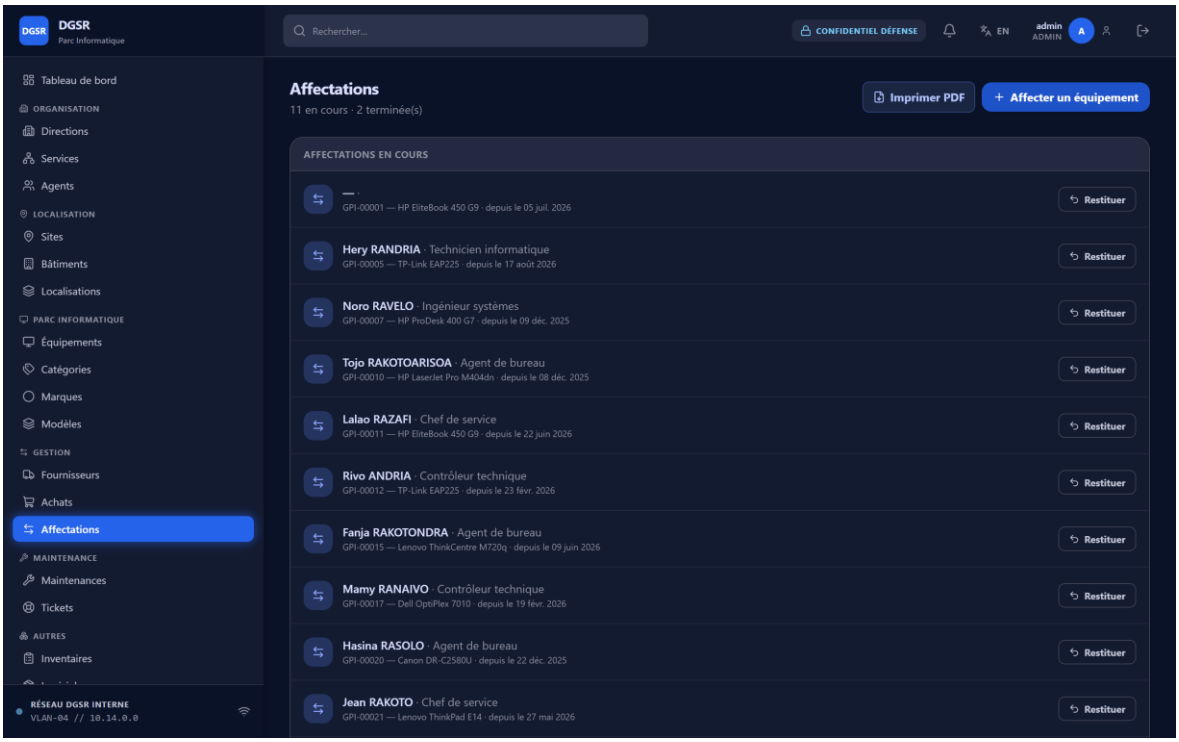



*Figure 11. Gestion des affectations en cours*

### *7.3.6. Maintenance et assistance*

Le module de maintenance assure le suivi des interventions préventives et curatives. Chaque intervention est rattachée à un équipement, à un type de maintenance et, si nécessaire, à un agent demandeur. Les statuts permettent de suivre le cycle : demandée, planifiée, en cours, terminée ou annulée. Les tickets d'assistance complètent ce dispositif : ils sont déclarés par les agents, qualifiés par priorité, puis traités par les techniciens avec un historique de commentaires.

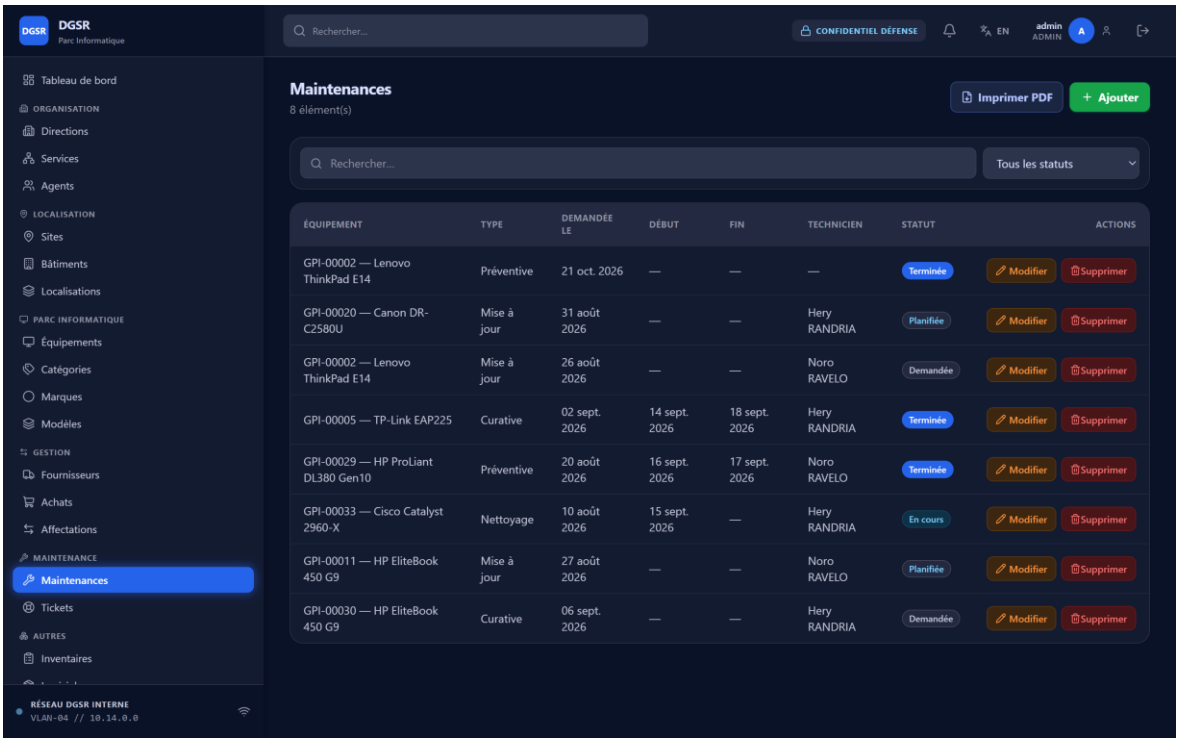



*Figure 12. Suivi des interventions de maintenance*

### *7.3.7. Inventaires*

Les campagnes d'inventaire permettent de vérifier la présence physique des équipements et de détecter les écarts. Une campagne possède une référence, une date de début, une date de fin prévisionnelle et un statut. Chaque ligne de constat indique si le matériel est présent, la date de vérification et une observation en cas d'écart.

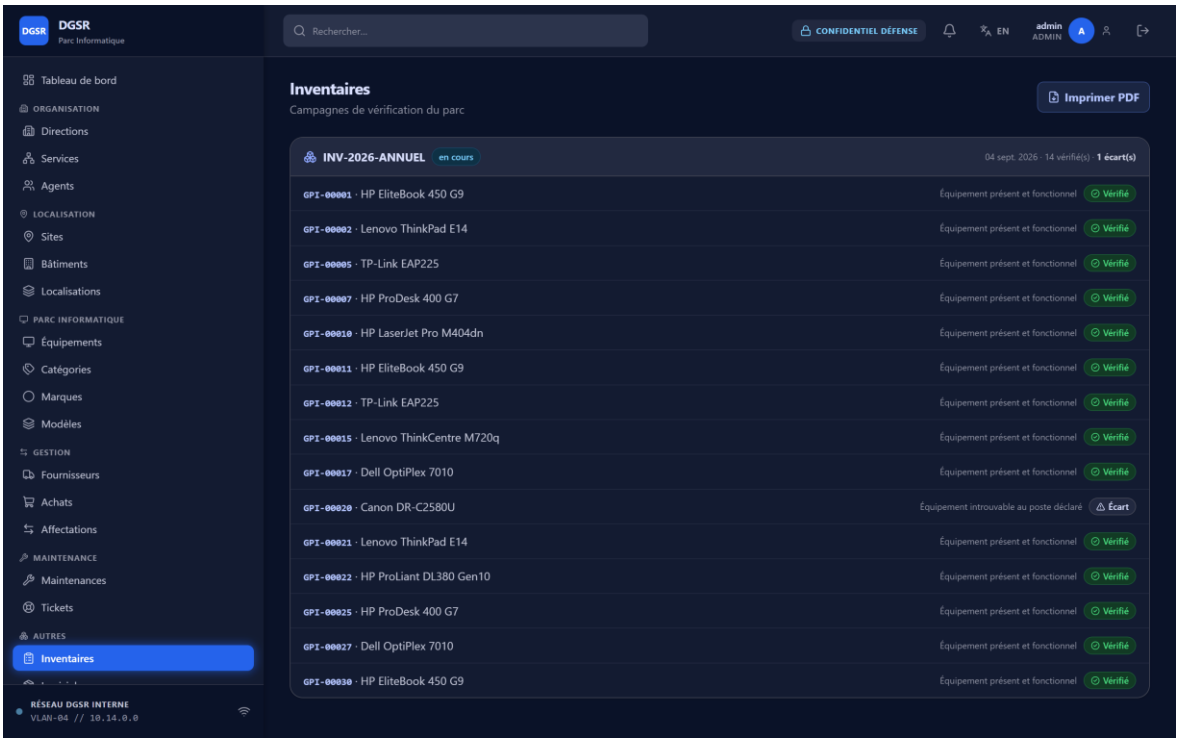



*Figure 13. Campagnes d'inventaire du parc*

### *7.3.8. Achats et fournisseurs*

Le module des achats conserve la traçabilité des acquisitions : référence, date, numéro de facture, bon de commande, montant et fournisseur. Les lignes de détail précisent la quantité, le prix unitaire et la date de livraison, et peuvent être rattachées à un équipement. Le répertoire des fournisseurs centralise les coordonnées et les références commerciales.

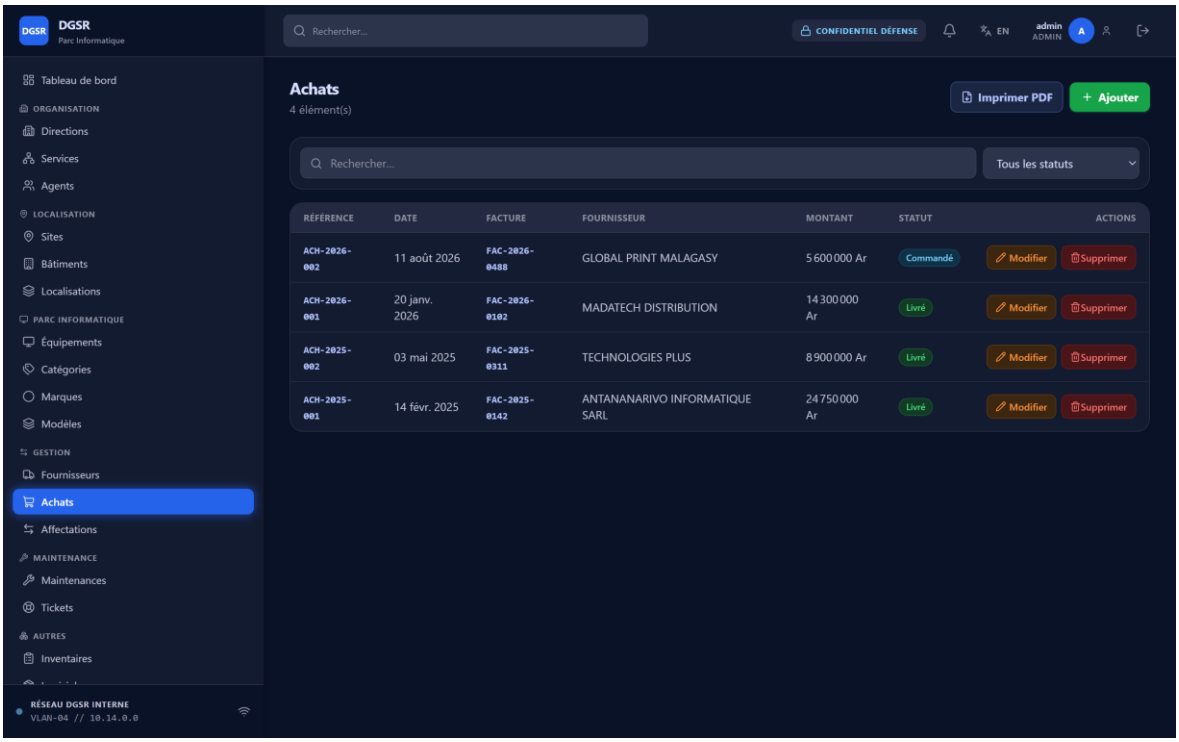



*Figure 14. Gestion des achats et des fournisseurs*

### *7.3.9. Logiciels et licences*

Le catalogue logiciel recense les applications utilisées, leur version et leur éditeur. Les licences permettent de suivre les dates d'acquisition et d'expiration, le nombre de postes autorisés et les références. Les installations relient un logiciel à un équipement, avec la date, la version installée et le statut. Une alerte est émise lorsqu'une licence approche de son échéance.

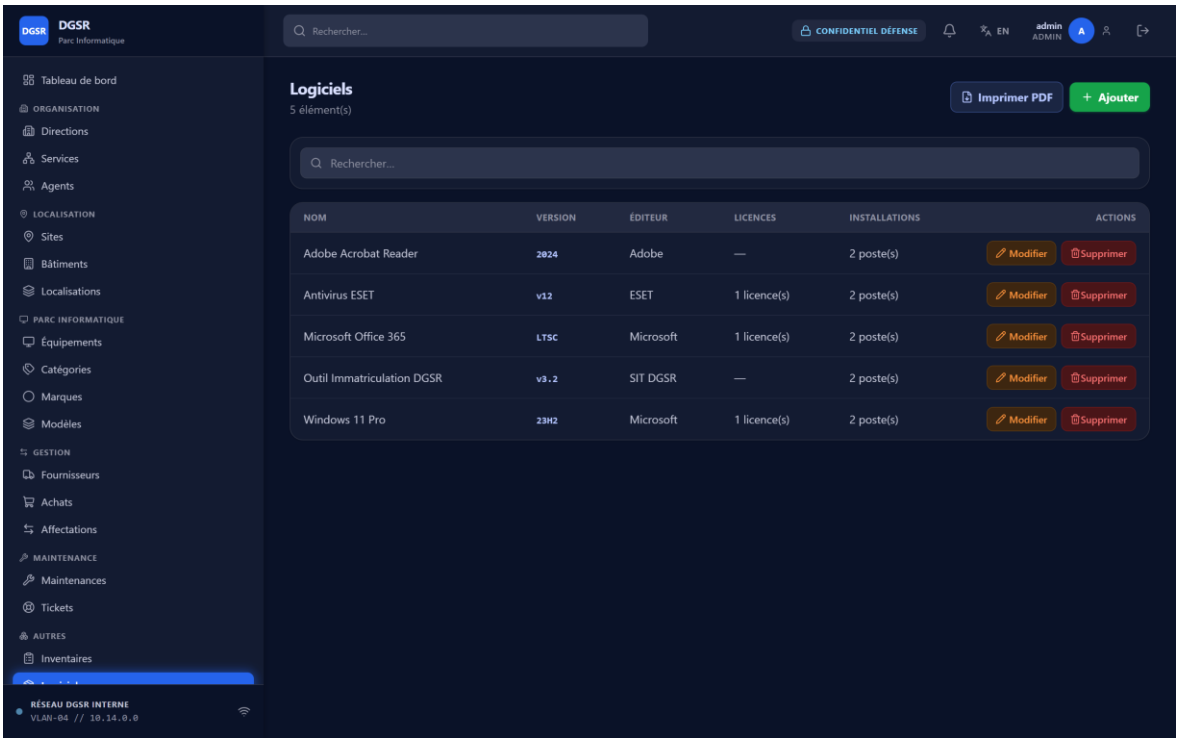



*Figure 15. Catalogue logiciel et suivi des licences*

### *7.3.10. Répertoire des agents*

La page des agents présente les fiches du personnel sous forme de cartes : nom, fonction, matricule, service, direction, téléphone et statut. La recherche s'effectue par matricule, nom ou fonction. Les agents peuvent être rattachés à un service et à une direction, ce qui permet ensuite de filtrer le parc par entité.

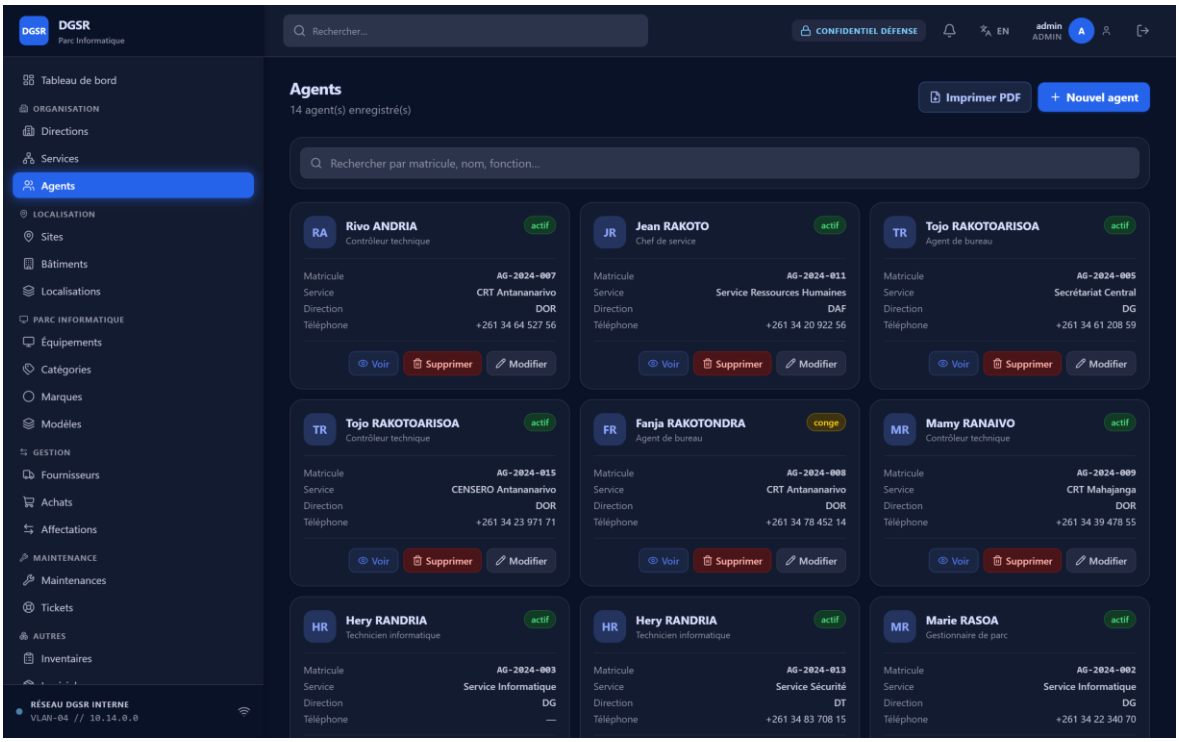



*Figure 16. Répertoire des agents de l'établissement*

### *7.3.11. Rapports et historiques*

La page des rapports propose trois vues : la synthèse du parc, les mouvements de matériel et le journal d'activité. Elle permet de choisir une période (année en cours, douze derniers mois, quatre-vingt-dix jours ou trente jours) et d'exporter le résultat en PDF. Les graphiques présentent les flux d'entrées et de sorties, les transactions par type et la répartition du parc par direction et par catégorie.

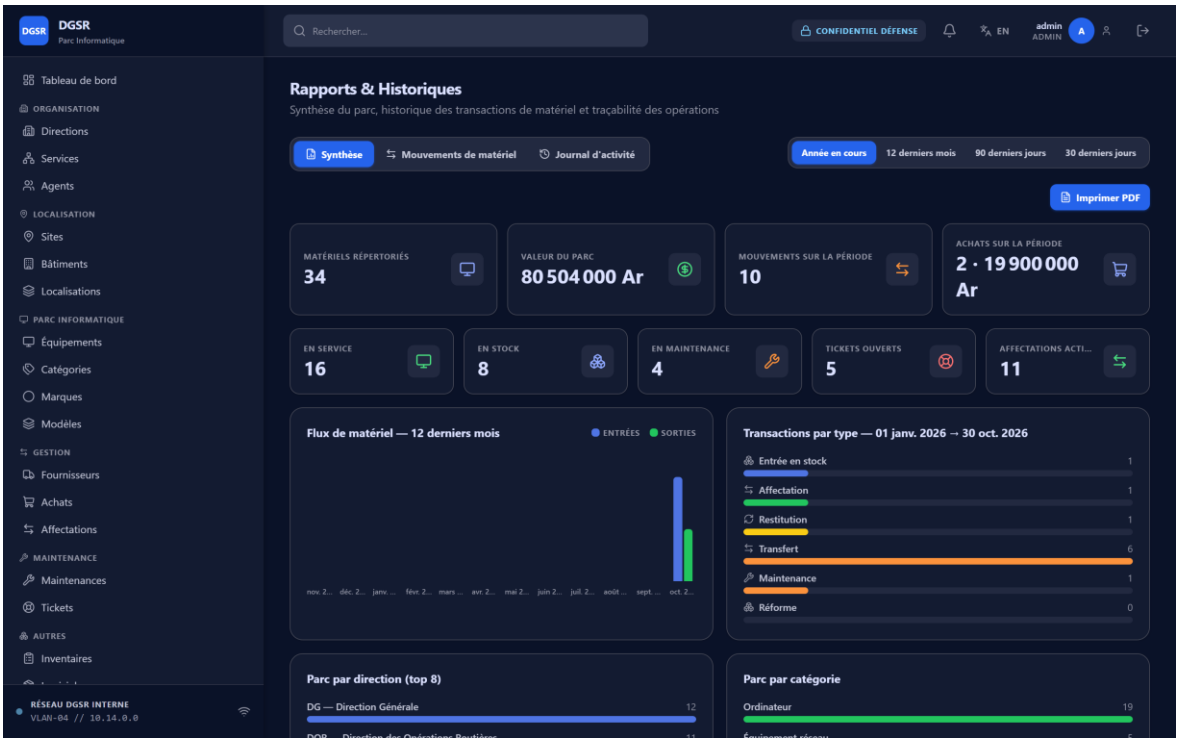



*Figure 17. Rapports et historiques du parc*

La synthèse met en évidence les indicateurs clés : matériels répertoriés, valeur du parc, mouvements sur la période, achats, équipements en service, en stock, en maintenance, tickets ouverts et affectations actives. Les exports PDF reprennent ces éléments dans un document officiel, prêt à être transmis à la hiérarchie.

## 7.4. Perspectives d'évolution

L'application couvre aujourd'hui l'essentiel des besoins du Service Informatique. Plusieurs évolutions sont envisagées pour les prochaines versions :

- **Étiquetage par QR code** : générer et imprimer des étiquettes, puis scanner un équipement depuis un téléphone pour ouvrir sa fiche.
- **Application mobile** : proposer une version adaptée aux agents en déplacement et aux interventions sur site.
- **Notifications par courriel** : alerter automatiquement les responsables lors d'une affectation, d'une panne critique ou d'une licence arrivant à échéance.
- **Tableau de bord décisionnel** : ajouter des indicateurs de coût de maintenance, de taux de disponibilité et de durée moyenne de résolution.
- **Annuaire LDAP / Active Directory** : synchroniser les comptes avec l'annuaire de l'établissement pour simplifier la gestion des accès.

- **Signature électronique** : intégrer un visa numérique sur les bons d'affectation et les fiches de restitution.
- **Sauvegarde automatisée** : planifier des sauvegardes quotidiennes de la base et des documents, avec test de restauration.
- **Journalisation avancée** : conserver l'historique détaillé des consultations et des exports, dans une logique d'audit.

# CONCLUSION GÉNÉRALE

Ce projet avait pour objectif principal de concevoir et de développer une application de gestion du parc informatique pour la Direction Générale de la Sécurité Routière, en s'appuyant sur des technologies modernes et adaptées aux besoins actuels des administrations. Tout au long de ce travail, une démarche méthodique a été suivie, depuis l'analyse des besoins jusqu'à la réalisation et à la présentation de l'application finale.

Dans un premier temps, l'étude préalable a permis de comprendre le contexte du projet et de définir les fonctionnalités essentielles du système. Le diagnostic de l'existant a mis en évidence les limites des fichiers tableurs : informations dispersées, absence de traçabilité, doublons et difficulté à produire des rapports fiables. La modélisation de la base de données et de l'architecture de l'application a ensuite joué un rôle clé dans l'organisation et la cohérence de la solution.

Le choix des technologies — Laravel pour le back-end, React pour l'interface, MySQL pour la gestion des données — s'est révélé pertinent en raison de leur robustesse, de leur évolutivité et de leur large adoption dans le monde professionnel. La méthode MERISE a fourni un cadre rigoureux pour passer des besoins exprimés aux tables de la base de données, puis aux traitements et aux interfaces.

La phase de développement a permis de mettre en pratique les connaissances théoriques acquises durant la formation. Le codage a été réalisé de manière structurée, en respectant les bonnes pratiques du framework : contrôleurs mutualisés, validation systématique, modèles Eloquent, middleware de rôles et migrations versionnées. Les différentes fonctionnalités ont été testées afin de garantir le bon fonctionnement de l'application et la fiabilité des données manipulées.

L'application obtenue centralise l'inventaire de trente-quatre équipements, le suivi de quatorze agents, les affectations, les achats, les maintenances, les tickets, les inventaires et les logiciels. Elle produit des indicateurs consolidés et des exports PDF destinés aux responsables. Elle répond ainsi aux objectifs fixés au début du projet et apporte au Service Informatique un outil de travail quotidien, plus fiable et plus rapide que les fichiers qu'il remplace.

Ce projet constitue une expérience enrichissante, tant sur le plan technique que méthodologique. Il m'a permis de renforcer mes compétences en conception de systèmes d'information, en développement d'API, en modélisation de bases de données et en intégration

d'interfaces. Il m'a également confronté aux contraintes réelles d'un projet : comprendre un besoin exprimé par des utilisateurs, arbitrer entre plusieurs solutions, corriger des anomalies et documenter le travail réalisé.

Plusieurs perspectives restent ouvertes : l'étiquetage par QR code, une application mobile, les notifications par courriel, la synchronisation avec l'annuaire de l'établissement et l'automatisation des sauvegardes. Ces évolutions permettront d'étendre la portée de l'outil et d'en faire un véritable système de pilotage du parc informatique.

# BIBLIOGRAPHIE

[1] TARDIEU H., ROCHFELD A., COLLETTI R., *La méthode Merise : principes et outils*, 1983, Éditions d'Organisation, Paris, 348 pages.

[2] DATE C. J., *Introduction aux systèmes de bases de données*, 2000, Addison-Wesley, Paris, 247 pages.

[3] GAMMA E., HELM R., JOHNSON R., VLISSIDES J., *Design Patterns : Elements of Reusable Object-Oriented Software*, 1994, Addison-Wesley, Reading, 395 pages.

[4] FREEMAN A., ROBSON E., *Head First JavaScript Programming*, 2014, O'Reilly Media, Sebastopol, 704 pages.

[5] NANOU B., *Architecture logicielle : concevoir des applications robustes et évolutives*, 2022, Dunod, Paris, 256 pages.

# WEBOGRAPHIE

[6] https://laravel.com/docs, Documentation officielle de Laravel, consultée le 10 novembre 2026.

[7] https://fr.react.dev, Documentation officielle de React, consultée le 10 novembre 2026.

[8] https://dev.mysql.com/doc/, Manuel de référence MySQL 8.4, consulté le 9 novembre 2026.

[9] https://tailwindcss.com/docs, Documentation de Tailwind CSS, consultée le 6 novembre 2026.

[10] https://vite.dev/guide/, Guide officiel de Vite, consulté le 6 novembre 2026.

[11] https://getcomposer.org/doc/, Documentation de Composer, consultée le 4 novembre 2026.

[12] https://git-scm.com/book/fr/v2, Pro Git, livre de référence en français, consulté le 3 novembre 2026.

[13] https://developer.mozilla.org/fr/, Ressources pour les développeurs web, consultées le 12 novembre 2026.

[14] https://www.php.net/manual/fr/, Manuel PHP, consulté le 8 novembre 2026.

[15] https://web.maths.unsw.edu.au/~lafaye/CCM/merise/concintro.htm, Cours de MERISE, consulté le 5 novembre 2026.

# GLOSSAIRE

| Terme | Définition |
|---|---|
| API | Interface permettant à deux applications de communiquer entre elles. Dans ce projet, l'API REST expose les données du parc au format JSON. |
| Back-end | Partie serveur de l'application, qui gère la logique métier, les règles de gestion et l'accès aux données. |
| Base de données | Ensemble structuré de données stockées et gérées par un système de gestion de base de données. |
| Composant | Élément d'interface réutilisable (tableau, formulaire, bouton) construit avec React. |
| Corbeille | Fonctionnalité de suppression logique permettant de restaurer un élément supprimé. |
| CRUD | Ensemble des opérations de création, lecture, mise à jour et suppression. |
| Eloquent | ORM de Laravel, qui fait correspondre les tables de la base à des classes PHP. |
| Front-end | Partie visible de l'application, avec laquelle l'utilisateur interagit. |
| Git | Système de gestion de versions distribué, utilisé pour suivre les modifications du code. |
| Intranet | Réseau local interne à une organisation, accessible uniquement depuis ses postes. |
| Jeton (token) | Chaîne générée à l'authentification et transmise à chaque requête pour prouver l'identité de l'utilisateur. |
| Laravel | Framework PHP open source fondé sur l'architecture MVC, utilisé pour le back-end. |
| MCD | Modèle Conceptuel des Données : représentation des entités, propriétés, relations et cardinalités. |
| MERISE | Méthode de conception des systèmes d'information fondée sur la modélisation des données et des traitements. |
| Middleware | Composant logiciel qui intercepte une requête HTTP pour appliquer un contrôle, par exemple la vérification d'un rôle. |
| Migration | Fichier versionné qui décrit une évolution du schéma de la base de données. |
| MLD | Modèle Logique des Données : traduction du MCD en relations (tables) avec clés primaires et étrangères. |
| MySQL | Système de gestion de base de données relationnelle open source utilisé pour stocker les données. |
| ORM | Object-Relational Mapping : mécanisme qui fait correspondre les objets du code aux tables de la base. |
| QR code | Code-barres bidimensionnel pouvant contenir une adresse ou un identifiant, utilisé pour étiqueter les équipements. |
| React | Bibliothèque JavaScript de construction d'interfaces utilisateur, utilisée pour le front-end. |
| Référentiel | Table de données de référence (catégories, marques, modèles, états) utilisée pour classer les équipements. |
| REST | Style d'architecture des services web fondé sur les ressources et les verbes HTTP. |
| Sanctum | Bibliothèque d'authentification de Laravel, utilisée pour générer et vérifier les jetons d'accès. |
| SGBD | Système de Gestion de Base de Données. |
| SPA | Single Page Application : application web qui charge une seule page et met à jour son contenu sans rechargement. |
| Vite | Outil de compilation et de serveur de développement pour les applications JavaScript. |

# ANNEXES

Les annexes rassemblent les documents techniques qui complètent le corps du rapport sans alourdir la présentation. Elles permettent de vérifier la traduction des modèles en code et l'organisation des points d'accès de l'application.

## Annexe A. Extrait du fichier de routes de l'API

Ce fichier déclare les points d'accès exposés par l'application et associe à chacun le contrôleur compétent, le middleware d'authentification et, lorsque l'opération modifie les données, le contrôle de rôle.

```
Route::post('/login', [AuthController::class, 'login'])->middleware('throttle:6,1');

Route::middleware('auth:sanctum')->group(function () {
    Route::get('/me', [AuthController::class, 'me']);
    Route::get('/dashboard', [DashboardController::class, 'stats']);

    // Ecriture reservee a l'administrateur et au gestionnaire de parc.
    $ecriture = 'role:admin,gestionnaire_parc';

    Route::apiResource('equipements', EquipementController::class)
        ->only(['index', 'show']);
    Route::apiResource('equipements', EquipementController::class)
        ->except(['index', 'show'])->middleware($ecriture);

    Route::apiResource('agents', AgentController::class)
        ->only(['index', 'show']);
    Route::apiResource('agents', AgentController::class)
        ->except(['index', 'show'])->middleware($ecriture);

    // Tout agent authentifie peut declarer une demande d'assistance.
    Route::apiResource('tickets', TicketController::class)
        ->only(['index', 'show', 'store']);
});
```

## Annexe B. Extrait du script de migration de la table equipements

Les tables sont créées par les migrations de Laravel. L'extrait ci-après montre la définition de la table centrale du parc, avec ses contraintes d'unicité, ses clés étrangères et son index sur le statut.

```
Schema::create('equipements', function (Blueprint $table) {
    $table->id('id_equipement');
    $table->string('code_inventaire', 50)->unique();
    $table->string('numero_serie', 100)->nullable()->unique();
    $table->string('designation', 200);
    $table->date('date_acquisition')->nullable();
    $table->decimal('prix_acquisition', 14, 2)->nullable();
    $table->date('garantie_debut')->nullable();
    $table->date('garantie_fin')->nullable();
    $table->enum('statut', [
        'en_service', 'en_stock', 'en_maintenance',
        'reforme', 'perdu', 'reserve',
    ])->default('en_stock');

    $table->foreignId('id_modele')->nullable()
        ->constrained('modeles', 'id_modele')->nullOnDelete();
    $table->foreignId('id_direction')->nullable()
        ->constrained('directions', 'id_direction')->nullOnDelete();
    $table->foreignId('id_localisation')->nullable()
        ->constrained('localisations', 'id_localisation')->nullOnDelete();

    $table->timestamps();
    $table->index('statut');
});
```

# TABLE DES MATIÈRES

CURRICULUM VITAE ..... I
PERSONNALITÉS ..... I
DIPLÔMES ET FORMATIONS ..... I
EXPÉRIENCES PROFESSIONNELLES ..... I
LANGUES ..... I
COMPÉTENCES ..... II
CENTRES D'INTÉRÊT ..... II
REMERCIEMENTS ..... V
LISTE DES FIGURES ..... VII
LISTE DES TABLEAUX ..... VIII
LISTE DES ABRÉVIATIONS ..... X
INTRODUCTION GÉNÉRALE ..... 1

**PARTIE I. PRÉSENTATIONS ..... 3**

Chapitre 1. Présentation de l'ENI ..... 4
1.1. Informations d'ordre général ..... 4
1.2. Missions et historique ..... 4
1.3. Organigramme institutionnel ..... 6
1.4. Domaines de spécialisation ..... 6
1.5. Architecture des formations pédagogiques ..... 7
1.6. Relations de l'ENI avec les organismes externes ..... 8
1.7. Débouchés professionnels ..... 8
1.8. Ressources humaines ..... 9
Chapitre 2. Présentation de la DGSR ..... 10
2.1. Statut et missions ..... 10
2.2. Organisation administrative ..... 10
2.3. Organigramme fonctionnel ..... 11
2.4. Valeurs et engagements ..... 11
2.5. Le Service Informatique ..... 12
Chapitre 3. Description du projet ..... 13
3.1. Formulation ..... 13
3.2. Objectif et besoins des utilisateurs ..... 13
3.3. Moyens nécessaires à la réalisation du projet ..... 14
3.3.1. Moyens humains ..... 14
3.3.2. Moyens matériels ..... 14
3.3.3. Moyens logiciels ..... 14
3.4. Résultats attendus ..... 15
3.5. Chronogramme de travail ..... 15

**PARTIE II. ANALYSE ET CONCEPTION ..... 17**

Chapitre 4. Analyse préalable ..... 18
4.1. Analyse de l'existant ..... 18
4.1.1. Organisation actuelle ..... 18
4.1.2. Inventaire des moyens matériels et logiciels ..... 18
4.2. Critique de l'existant ..... 19
4.3. Conception avant-projet ..... 19
4.3.1. Proposition de solutions ..... 19
4.3.2. Méthodes de conception et outils ..... 20
4.3.3. Choix du système de gestion de base de données ..... 20
4.3.4. Choix du framework back-end ..... 21
4.3.5. Choix de la technologie front-end ..... 22
4.3.6. Environnement de développement et outils ..... 22
Chapitre 5. Analyse conceptuelle ..... 24
5.1. Présentation de la méthode MERISE ..... 24

5.2. Dictionnaire des données ..... 25
a) Entité AGENT ..... 25
b) Entité ÉQUIPEMENT ..... 25
c) Entité AFFECTATION ..... 26
d) Entité TICKET ..... 26
e) Entité MAINTENANCE ..... 27
f) Entité ACHAT ..... 27
g) Entité LOGICIEL ..... 27
5.3. Règles de gestion ..... 28
5.4. Diagramme de flux ..... 28
5.4.1. Acteurs du système ..... 29
5.4.2. Détail des flux d'informations ..... 29
5.5. Modélisation des données ..... 30
5.5.1. Modèle Conceptuel des Données (MCD) ..... 30
5.5.2. Modèle Logique des Données (MLD) ..... 32
5.5.3. Modèle Conceptuel des Traitements (MCT) ..... 35
5.5.4. Modèle Organisationnel des Traitements (MOT) ..... 37

**PARTIE III. RÉALISATION ..... 39**

Chapitre 6. Mise en place de l'environnement de développement ..... 40
6.1. Installation et configuration des outils ..... 40
a) Visual Studio Code ..... 40
b) WampServer, Apache et PHP ..... 40
c) Composer et Laravel ..... 41
d) Node.js, npm, Vite et Tailwind CSS ..... 41
e) MySQL et phpMyAdmin ..... 42
f) Git ..... 42
6.2. Architecture de l'application ..... 43
6.2.1. Couche présentation ..... 43
6.2.2. Couche métier ..... 43
6.2.3. Couche données ..... 44
6.2.4. Communication entre les couches ..... 44
6.3. Structure du projet ..... 44
6.4. Déploiement en mode intranet ..... 45
Chapitre 7. Développement de l'application ..... 47
7.1. Création de la base de données ..... 47
7.2. Codage de l'application ..... 48
7.2.1. Les modèles Eloquent ..... 48
7.2.2. Les contrôleurs et l'API REST ..... 48
7.2.3. Authentification, rôles et sécurité ..... 49
7.2.4. Application des règles de gestion ..... 50
7.2.5. Exports PDF et documents officiels ..... 51
7.2.6. L'interface React ..... 51
7.2.7. Tests et validation ..... 52
7.2.8. Difficultés rencontrées et solutions ..... 52
7.3. Présentation de l'application ..... 53
7.3.1. Page d'authentification ..... 53
7.3.2. Tableau de bord ..... 54
7.3.3. Gestion des équipements ..... 55
7.3.4. Fiche détaillée d'un équipement ..... 56
7.3.5. Affectations et restitutions ..... 57
7.3.6. Maintenance et assistance ..... 58
7.3.7. Inventaires ..... 59
7.3.8. Achats et fournisseurs ..... 60
7.3.9. Logiciels et licences ..... 61
7.3.10. Répertoire des agents ..... 62
7.3.11. Rapports et historiques ..... 63
7.4. Perspectives d'évolution ..... 64

CONCLUSION GÉNÉRALE ..... 66
BIBLIOGRAPHIE ..... 68
WEBOGRAPHIE ..... 69
GLOSSAIRE ..... 70
ANNEXES ..... 71
Annexe A. Extrait du fichier de routes de l'API ..... 71
Annexe B. Extrait du script de migration de la table equipements ..... 72
RÉSUMÉ ..... 76
ABSTRACT ..... 77

# RÉSUMÉ

Ce projet porte sur la conception et le développement d'une application web intranet de gestion du parc informatique (GPI) pour la Direction Générale de la Sécurité Routière. L'objectif est de remplacer les fichiers tableurs dispersés par une plateforme centralisée, fiable et traçable.

La démarche adoptée s'appuie sur la méthode MERISE : analyse de l'existant, dictionnaire des données, règles de gestion, diagrammes de flux, modèle conceptuel des données, modèle logique, modèle conceptuel des traitements et modèle organisationnel. L'application a été développée avec Laravel pour l'API REST, React pour l'interface et MySQL pour le stockage.

La plateforme couvre l'inventaire des équipements, les affectations et restitutions, les maintenances et tickets d'assistance, les achats et fournisseurs, les logiciels et licences, les campagnes d'inventaire, la traçabilité des mouvements et la production de rapports exportables en PDF. Elle repose sur quatre rôles (administrateur, gestionnaire de parc, technicien, utilisateur) et sur des règles de gestion vérifiées par des tests.

Les résultats obtenus montrent que l'application répond aux besoins définis : le parc de trente-quatre équipements et les quatorze agents sont centralisés, les affectations sont suivies et les responsables disposent d'indicateurs à jour. Ce travail a permis de mettre en pratique les connaissances acquises durant la formation et de développer des compétences en conception, développement, sécurisation et déploiement d'applications.

**Mots-clés** : affectation, application intranet, gestion du parc informatique, inventaire, Laravel, maintenance, MERISE, MySQL.

# ABSTRACT

This project deals with the design and development of an intranet web application for IT asset management (GPI) at the General Directorate for Road Safety. The aim is to replace scattered spreadsheet files with a centralized, reliable and auditable platform.

The approach is based on the MERISE method: analysis of the existing system, data dictionary, business rules, data flow diagrams, conceptual data model, logical model, conceptual process model and organizational process model. The application was built with Laravel for the REST API, React for the user interface and MySQL for data storage.

The platform covers equipment inventory, assignments and returns, maintenance and support tickets, purchases and suppliers, software and licences, inventory campaigns, movement tracking and PDF reports. It relies on four roles (administrator, asset manager, technician, user) and on business rules verified by tests.

The results show that the application meets the defined requirements: the thirty-four pieces of equipment and fourteen agents are centralized, assignments are tracked and managers have up-to-date indicators. This work made it possible to put theoretical knowledge into practice and to develop skills in design, development, security and deployment.

**Keywords**: assignment, intranet application, inventory, IT asset management, Laravel, maintenance, MERISE, MySQL.